

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a0b422e7d58b898c5.jsonl`  
- SHA-256: `bb700fb6cbe2a69c719008ee6617369fc1022716c71e6d3f2dab31e262a7d84`  
- Source modified: `2026-07-02T02:55:03+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:47:11.981Z)

CONTEXT (read carefully):

- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidullu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/O3/O4) · docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog M4/P0-rename/O1/O2 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: full audit of sports-data-collector. Repo: C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector.

Small Python repo (label/corpus collector; recent PRs #50 skipped-row warning, #53 busy_timeout). Audit: code quality, test coverage + CI state (gh run list), error handling on ingestion paths, schema/migration hygiene, doc accuracy. SPECIAL: the working tree is DIRTY - a modified .gitignore + an UNTRACKED nested directory sports-data-collector/sports-data-collector/. INSPECT (read-only!) what that nested dir contains (ls -la, file sizes, is it a stray clone? generated data?), diff the .gitignore change (git diff .gitignore), and report

exactly what it is + a safe disposition recommendation. Do NOT delete or modify anything.

2. user (2026-07-02T02:47:18.875Z)

On branch master

Your branch is up to date with 'origin/master'.

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: .gitignore

Untracked files:

(use "git add <file>..." to include in what will be committed)

sports-data-collector/

no changes added to commit (use "git add" and/or "git commit -a")

adcd206 fix(db): set SQLite busy_timeout so concurrent writers wait, not error (P9) (#53)

6f67d65 feat(import): callable import_annotations + a maturity gate (P9) (#52)

c281f82 feat(export): add source file MD5 to the eval/training manifest (P9) (#51)

a152cf5 fix(cvat): warn on rows silently skipped by load_canonical_csv (#50)

75bb0fa docs: archive CODE_HEALTH_PLAN ? project DONE (#49)

a09a000 refactor(manifests): typed pydantic models for the export manifest (A5) (#48)

6678fb0 refactor(curate): lift sys.exit out of _guard_import into CurationError (A2) (#47)

9e56056 fix(reporting): make FAQ path configurable via env var (A6) (#46)

0ddb022 docs: Tier D triage (D8) + NEXT_STEPS for parked A1/A3 (#45)

c7c4f68 refactor(config): promote validation thresholds to config (Tier C) (#43)

warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

diff --git a/.gitignore b/.gitignore

index 52dac92..ac1420f 100644

--- a/.gitignore

+++ b/.gitignore

@@ -53,3 +53,7 @@ data/temp/

Local DB backups (e.g. data/sports_metadata.backup-YYYY-MM-DD.db) ? never commit

data/*.backup-*.db

+

+# Model artifact bundles: weights are NEVER committed (private Releases); the

+# reviewable surface is each bundle's manifest.json (ADR-008).

+artifacts/

3. user (2026-07-02T02:47:20.472Z)

total 113

drwxr-xr-x 1 avidu 197609 0 Jun 29 15:27 .

drwxr-xr-x 1 avidu 197609 0 Jun 29 20:32 ..

-rw-r--r-- 1 avidu 197609 53248 Jun 10 11:35 .coverage

drwxr-xr-x 1 avidu 197609 0 Jul 2 08:17 .git

-rw-r--r-- 1 avidu 197609 264 May 26 01:26 .gitattributes

drwxr-xr-x 1 avidu 197609 0 Jun 10 13:07 .github

-rw-r--r-- 1 avidu 197609 1292 Jun 10 15:53 .gitignore

drwxr-xr-x 1 avidu 197609 0 May 25 23:59 .pytest_cache

drwxr-xr-x 1 avidu 197609 0 Jun 15 14:57 .ruff_cache

-rw-r--r-- 1 avidu 197609 9583 Jun 2 22:52 README.md

drwxr-xr-x 1 avidu 197609 0 Jun 29 14:48 __pycache__

-rwxr-xr-x 1 avidu 197609 7602 Jun 29 14:55 cleanup.py

drwxr-xr-x 1 avidu 197609 0 Jun 29 14:55 config

drwxr-xr-x 1 avidu 197609 0 Jun 29 14:55 data

drwxr-xr-x 1 avidu 197609 0 Jun 29 14:55 docs

-rw-r--r-- 1 avidu 197609 1451 Jun 29 14:55 pyproject.toml

drwxr-xr-x 1 avidu 197609 0 Jun 19 21:38 sports-data-collector

drwxr-xr-x 1 avidu 197609 0 Jun 7 13:10 src

```

drwxr-xr-x 1 avidu 197609      0 Jul  2 08:13 tests
=== nested ===
total 53
drwxr-xr-x 1 avidu 197609      0 Jun 19 21:38 .
drwxr-xr-x 1 avidu 197609      0 Jun 29 15:27 ..
drwxr-xr-x 1 avidu 197609      0 Jun 29 14:41 .git
-rw-r--r-- 1 avidu 197609    264 Jun  2 22:13 .gitattributes
drwxr-xr-x 1 avidu 197609      0 Jun 19 21:38 .github
-rw-r--r-- 1 avidu 197609   1139 Jun 19 21:38 .gitignore
-rw-r--r-- 1 avidu 197609   9583 Jun 19 21:38 README.md
-rwxr-xr-x 1 avidu 197609   6802 Jun 19 21:38 cleanup.py
drwxr-xr-x 1 avidu 197609      0 Jun  2 22:13 config
drwxr-xr-x 1 avidu 197609      0 Jun 19 21:38 data
drwxr-xr-x 1 avidu 197609      0 Jun 19 21:38 docs
-rw-r--r-- 1 avidu 197609   1211 Jun 19 21:38 pyproject.toml
drwxr-xr-x 1 avidu 197609      0 Jun 19 21:38 src
drwxr-xr-x 1 avidu 197609      0 Jun 19 21:38 tests

```

4. user (2026-07-02T02:47:30.357Z)

```

origin https://github.com/avidullu/sports-data-collector.git (fetch)
origin https://github.com/avidullu/sports-data-collector.git (push)
031e410 feat(annotation): richer Roora schema - court + confidence + capture report + IAA (#29)
3016618 feat(curate): --encoder nvenc (GPU) option for proxy + batch transcode (#28)
04a3049 feat(curate): preflight gate for prep-annotation-batch (dup/empty/VFR checks) (#27)
0a0cff5 docs(runbook): point operators at the canonical vendor guidelines Google Doc (#26)
ca61405 docs(annotation): ROORA_RUNBOOK - the operator e2e (send -> receive -> golden labels)
(#25)
* master
  remotes/origin/HEAD -> origin/master
  remotes/origin/master
---size---
1.7M   .
811K   .git
---data dir---
data/mock_local_badminton_anno.json
data/roora_badminton_canonical.csv
data/roora_badminton_issues.txt
data/sports_metadata.db
data/staging_coachai_match.csv
data/validation/pilot_badminton_01_report.json
data/validation/pilot_badminton_01_report.md
data/validation/pilot_badminton_01_review.csv
---diff vs parent---
031e410 feat(annotation): richer Roora schema - court + confidence + capture report + IAA (#29)

```

5. user (2026-07-02T02:47:30.723Z)

```

src/cli/curate.py
src/cli/ingest.py
src/cli/normalize_annotations.py
src/core/batch_preflight.py
src/core/config.py
src/core/db.py
src/core/downloader.py
src/core/manifests.py
src/core/parsing.py
src/core/proxy.py
src/core/schemas/badminton.py
src/core/schemas/base.py
src/core/schemas/cricket.py
src/core/schemas/pickleball.py
src/core/schemas/tabletennis.py
src/core/schemas/tennis.py
src/core/schemas/_mixins.py

```

```
src/core/validator.py
src/crawlers/badminton_crawler.py
src/parsers/badminton/coachai.py
src/parsers/base_parser.py
src/parsers/cvat/rally_cvat.py
src/parsers/cvat/__init__.py
src/reporting/adapter.py
src/reporting/__init__.py
tests/test_batch_preflight.py
tests/test_cleanup.py
tests/test_cli.py
tests/test_config.py
tests/test_crawler_tempfile.py
tests/test_cvat_parser.py
tests/test_cvat_robustness.py
tests/test_db.py
tests/test_db_pragmas.py
tests/test_downloader.py
tests/test_import_guard.py
tests/test_license_consistency.py
tests/test_licensing_gate.py
tests/test_local_import.py
tests/test_manifests.py
tests/test_normalize_annotations.py
tests/test_parser.py
tests/test_parsing.py
tests/test_proxy.py
tests/test_reporting.py
tests/test_reporting_faq.py
tests/test_reporting_noraise.py
tests/test_schemas.py
tests/test_validator.py
---
```

name: CI

```
on:
  push:
    branches: [master]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"
      - name: Run tests
        run: python -m pytest -q
```

6. user (2026-07-02T02:47:47.609Z)

```
data/mock_local_badminton_anno.json
data/roora_badminton_canonical.csv
data/roora_badminton_issues.txt
data/sports_metadata.db
data/staging_coachai_match.csv
data/validation/pilot_badminton_01_report.json
data/validation/pilot_badminton_01_report.md
data/validation/pilot_badminton_01_review.csv
```

```
---artifacts dir exists?---
no artifacts dir
---ADR-008?---
DECISIONS.md
INGESTION_PIPELINE.md
MANUAL_INGESTION_POSITIONING.md
NEXT_STEPS.md
PIPELINE_ROLE.md
annotation
archives
```

7. user (2026-07-02T02:47:51.584Z)

```
completed      success fix(db): set SQLite busy_timeout so concurrent writers wait, not erro?
CI      master  push      28518501041      18s      2026-07-01T12:45:50Z
completed      success fix(db): SQLite busy_timeout so concurrent writers wait, not error (P9)
CI      feat/db-busy-timeout  pull_request  28518467031      22s      2026-07-01T12:45:14Z
completed      success feat(import): callable import_annotations + a maturity gate (P9) (#52)
CI      master  push      28508171622      23s      2026-07-01T09:37:50Z
completed      success feat(import): callable import_annotations + a maturity gate (P9)
CI      feat/import-annotations-fn  pull_request  28508131311      19s
2026-07-01T09:37:06Z
completed      success feat(export): add source file MD5 to the eval/training manifest (P9) ?
CI      master  push      28506889883      22s      2026-07-01T09:15:35Z
completed      success feat(export): add source file MD5 to the eval/training manifest (P9)
CI      feat/export-manifest-md5  pull_request  28506853998      22s
2026-07-01T09:15:01Z
completed      success fix(cvat): warn on rows silently skipped by load_canonical_csv (#50)
CI      master  push      28361803082      21s      2026-06-29T09:20:57Z
completed      success fix(cvat): warn on rows silently skipped by load_canonical_csv CI
fix/cvat-load-canonical-warn  pull_request  28361748170      16s      2026-06-29T09:20:04Z
---
---
```

8. user (2026-07-02T02:47:51.590Z)

```
1      [build-system]
2      requires = ["setuptools>=61.0.0"]
3      build-backend = "setuptools.build_meta"
4
5      [project]
6      name = "sports-data-collector"
7      version = "0.1.0"
8      description = "A collector and curator for sports video datasets with sport-specific
schemas and license checks."
9      readme = "README.md"
10     requires-python = ">=3.9"
11     dependencies = [
12         "pydantic>=2.0.0",
13         "pyyaml>=6.0",
14         "tabulate>=0.9.0",
15         # Hardens CVAT XML/zip parsing against entity-expansion / billion-laughs
16         # attacks (see docs/CODE_HEALTH_PLAN.md §7 Tier C). BSD-2-Clause ?
17         # commercially-clean per the project's licensing rule.
18         "defusedxml>=0.7.0",
19     ]
20
21     [project.optional-dependencies]
22     # Video acquisition (`fetch` / crawler). Also requires the external binaries
23     # ffprobe (FFmpeg) and, for YouTube high-res, a JS runtime such as deno ?
24     # neither is pip-installable; see README "External requirements".
25     fetch = [
26         "yt-dlp>=2024.1.0"
27     ]
28     dev = [
```

```

29         "pytest>=7.0.0",
30         "pytest-cov>=4.0.0"
31     ]
32     # Per-video observability reports (SOFT dependency: ingestion degrades to a no-op
33     # if absent). Shared, permissively-licensed report toolkit. Pin a tag once the
34     # private repo is published, e.g.:
35     #   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
36     # For local development install it editable from a sibling checkout:
37     #   pip install -e ../sports-obsreport --no-build-isolation
38     report = [
39         "obsreport>=0.1.3",
40     ]
41

```

9. user (2026-07-02T02:48:07.228Z)

```

nested-HEAD IS ancestor of current master
clean-check-done
a152cf5 fix(cvat): warn on rows silently skipped by load_canonical_csv (#50)
75bb0fa docs: archive CODE_HEALTH_PLAN ? project DONE (#49)
a09a000 refactor(manifests): typed pydantic models for the export manifest (A5) (#48)

```

10. user (2026-07-02T02:48:08.431Z)

```

9f1e792 fix(rally_cvat): harden XML parsing with defusedxml (T7/OQ5) (#42)
9e6f204 feat(corpus): ingest the n=6 owned golden corpus (262 rallies, Proprietary/commercial)
(#19)
e4dd8f1 chore(corpus): reclassify 23 ShuttleSet broadcast videos as non-commercial (owner-
approved) (#17)
e380197 data: upgrade 22 ShuttleSet matches to 1080p; null stale shuttleset_12 path
ed87652 feat: throttle + per-video retry in fetch; skip placeholder URLs
9f6f7d4 fix: default to 'default' player client (web_safari is SABR-crippled)
8e7bbdf fix: reliable best-resolution downloads (EJS solver, deno path, cookies)
854b077 Initial commit: sports data collector & curator
total 466
drwxr-xr-x 1 avidu 197609      0 Jun 29 14:55 .
drwxr-xr-x 1 avidu 197609      0 Jun 29 15:27 ..
drwxr-xr-x 1 avidu 197609      0 May 26 02:57 eval_export
-rw-r--r-- 1 avidu 197609    367 May 25 23:59 mock_local_badminton_anno.json
-rw-r--r-- 1 avidu 197609   1395 Jun  7 14:29 roora_badminton_canonical.csv
-rw-r--r-- 1 avidu 197609    827 Jun  7 14:29 roora_badminton_issues.txt
-rw-r--r-- 1 avidu 197609 200704 Jun 10 13:35 sports_metadata.backup-2026-06-10.db
-rw-r--r-- 1 avidu 197609 233472 Jun 29 14:55 sports_metadata.db
-rw-r--r-- 1 avidu 197609    148 May 26 00:00 staging_coachai_match.csv
drwxr-xr-x 1 avidu 197609      0 Jun  2 22:36 temp
drwxr-xr-x 1 avidu 197609      0 Jun  7 14:29 validation
drwxr-xr-x 1 avidu 197609      0 May 26 02:49 videos

```

Normalize line endings: store text as LF in the repo, check out native.

```
* text=auto
```

Treat data/media as binary ? never line-ending-convert or text-diff these.

```

*.db binary
*.sqlite binary
*.sqlite3 binary
*.mp4 binary
*.png binary
*.jpg binary
*.jpeg binary

```

11. user (2026-07-02T02:48:40.020Z)

```

1     import sqlite3
2     import os

```

```

3     import logging
4     from contextlib import contextmanager
5     from enum import Enum
6     from typing import List, Dict, Any, Optional, Type, TypeVar
7     from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
8     from src.core.schemas.badminton import BadmintonRally
9     from src.core.schemas.tennis import TennisPoint
10    from src.core.schemas.cricket import CricketDelivery
11    from src.core.schemas.pickleball import PickleballRally
12    from src.core.schemas.tabletennis import TableTennisRally
13
14    logger = logging.getLogger(__name__)
15
16    # Maps each sport to its annotation table and the column(s) that order segments
17    # chronologically. All child tables expose (start_time, end_time), so a
18    # single generic reader can serve every sport ? adding a new sport is one entry
19    # here plus its CREATE TABLE, with no new query code.
20    SEGMENT_TABLES = {
21        SportType.BADMINTON: ("badminton_rallies", "rally_number"),
22        SportType.TENNIS: ("tennis_points", "point_number"),
23        SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
24        SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
25        SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
26    }
27
28
29    class SportsDatabase:
30        def __init__(self, db_path: str):
31            self.db_path = db_path
32            # Ensure target directory exists
33            os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
34            self._init_db()
35
36        @contextmanager
37        def _get_connection(self):
38            """Yield a SQLite connection and always close it.
39
40            Used as ``with self._get_connection() as conn:``. A plain
41            ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
42            transaction but does NOT close the connection, which leaks handles and
43            raises ResourceWarnings. This wrapper guarantees the connection is closed
44            on exit; writers still commit explicitly inside the block.
45            """
46            conn = sqlite3.connect(self.db_path)
47            conn.row_factory = sqlite3.Row
48            # SQLite disables foreign keys per-connection by default; enable so the
49            # ON DELETE CASCADE declarations on the child tables actually fire.
50            conn.execute("PRAGMA foreign_keys = ON")
51            # Wait (up to 5s) for a competing writer's lock instead of failing immediately
52            with
53                # "database is locked" ? makes concurrent access robust (a sibling CLI, or a
54                # web-driven import_annotations writer) rather than racy. Writers still commit
55                explicitly.
56                conn.execute("PRAGMA busy_timeout = 5000")
57                try:
58                    yield conn
59                finally:
60                    conn.close()
61
62        def _init_db(self):
63            with self._get_connection() as conn:
64                cursor = conn.cursor()
65
66                # 1. Create common videos table

```

```

65 cursor.execute("""
66 CREATE TABLE IF NOT EXISTS videos (
67     video_id TEXT PRIMARY KEY,
68     sport TEXT NOT NULL,
69     video_url TEXT,
70     local_path TEXT,
71     license_type TEXT NOT NULL,
72     license_url TEXT,
73     annotation_license TEXT,
74     is_commercial INTEGER NOT NULL DEFAULT 0,
75     status TEXT NOT NULL DEFAULT 'staging',
76     match_name TEXT NOT NULL,
77     fps REAL,
78     resolution TEXT
79 )
80 """)
81
82 # 2. Create badminton_rallies table
83 cursor.execute("""
84 CREATE TABLE IF NOT EXISTS badminton_rallies (
85     video_id TEXT NOT NULL,
86     rally_number INTEGER NOT NULL,
87     start_time REAL NOT NULL,
88     end_time REAL NOT NULL,
89     score_before TEXT,
90     score_after TEXT,
91     shots_count INTEGER,
92     ending_reason TEXT,
93     last_shot_outcome TEXT,
94     PRIMARY KEY (video_id, rally_number),
95     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
96 )
97 """)
98
99 # 3. Create tennis_points table
100 cursor.execute("""
101 CREATE TABLE IF NOT EXISTS tennis_points (
102     video_id TEXT NOT NULL,
103     point_number INTEGER NOT NULL,
104     start_time REAL NOT NULL,
105     end_time REAL NOT NULL,
106     set_score TEXT,
107     game_score TEXT,
108     server TEXT,
109     ending_type TEXT,
110     PRIMARY KEY (video_id, point_number),
111     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
112 )
113 """)
114
115 # 4. Create cricket_deliveries table
116 cursor.execute("""
117 CREATE TABLE IF NOT EXISTS cricket_deliveries (
118     video_id TEXT NOT NULL,
119     over_num INTEGER NOT NULL,
120     ball_num INTEGER NOT NULL,
121     start_time REAL NOT NULL,
122     end_time REAL NOT NULL,
123     batsman TEXT,
124     bowler TEXT,
125     runs INTEGER,
126     wicket INTEGER DEFAULT 0,
127     extras INTEGER DEFAULT 0,
128     PRIMARY KEY (video_id, over_num, ball_num),
129     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE

```



```

130         )
131     """)
132
133     # 5. Create pickleball_rallies table
134     cursor.execute("""
135     CREATE TABLE IF NOT EXISTS pickleball_rallies (
136         video_id TEXT NOT NULL,
137         rally_number INTEGER NOT NULL,
138         start_time REAL NOT NULL,
139         end_time REAL NOT NULL,
140         score_before TEXT,
141         score_after TEXT,
142         shots_count INTEGER,
143         ending_reason TEXT,
144         last_shot_outcome TEXT,
145         PRIMARY KEY (video_id, rally_number),
146         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
147     )
148     """)
149
150     # 6. Create tabletennis_rallies table
151     cursor.execute("""
152     CREATE TABLE IF NOT EXISTS tabletennis_rallies (
153         video_id TEXT NOT NULL,
154         rally_number INTEGER NOT NULL,
155         start_time REAL NOT NULL,
156         end_time REAL NOT NULL,
157         score_before TEXT,
158         score_after TEXT,
159         shots_count INTEGER,
160         ending_reason TEXT,
161         last_shot_outcome TEXT,
162         PRIMARY KEY (video_id, rally_number),
163         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
164     )
165     """)
166
167     conn.commit()
168     self._migrate(cursor)
169     conn.commit()
170
171     #: The current schema version. Bump when adding a migration step below.
172     SCHEMA_VERSION = 1
173
174     def _migrate(self, cursor) -> None:
175         """Run the versioned migration ladder via PRAGMA user_version.
176
177         Each step is gated on the DB's current version and bumps it after applying.
178         The column-sniffing inside each step stays (so a DB that predates versioning
179         upgrades transparently on first open), but is now scoped and ordered. Future
180         migrations add a new step + bump SCHEMA_VERSION ? no more ad-hoc ALTERs.
181         See docs/CODE_HEALTH_PLAN.md §7 Tier C.
182         """
183         current = cursor.execute("PRAGMA user_version").fetchone()[0]
184         if current >= self.SCHEMA_VERSION:
185             return # up to date
186
187         # --- v1: add last_shot_outcome to rally tables + split video/annotation license
188         if current < 1:
189             # add last_shot_outcome to rally tables that predate the column. CREATE
190             TABLE
191             # IF NOT EXISTS won't alter an existing table, so do it explicitly and
192             # idempotently (skip if the column is already there).
193             for tbl in ("badminton_rallies", "pickleball_rallies",
194 "tabletennis_rallies"):

```

```

193         existing = [r[1] for r in cursor.execute(f"PRAGMA
table_info({tbl})").fetchall()]
194         if "last_shot_outcome" not in existing:
195             cursor.execute(f"ALTER TABLE {tbl} ADD COLUMN last_shot_outcome
TEXT")
196
197         # split the conflated single license into video vs annotation license.
198         # annotation_license records the dataset's annotation rights (e.g.
ShuttleSet
199         # MIT) for provenance; license_type stays the VIDEO's rights, which gates
200         # commercial use.
201         video_cols = [r[1] for r in cursor.execute("PRAGMA
table_info(videos)").fetchall()]
202         if "annotation_license" not in video_cols:
203             cursor.execute("ALTER TABLE videos ADD COLUMN annotation_license TEXT")
204
205             cursor.execute(f"PRAGMA user_version = {self.SCHEMA_VERSION}")
206
207     def insert_video(self, video: VideoMetadata) -> bool:
208         """Insert a video entry, or update it in place if it already exists.
209
210         Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
211         existing row, which (with foreign keys enabled) would cascade-delete the
212         video's child annotations. UPSERT mutates the row in place and leaves
213         children untouched.
214         """
215         with self._get_connection() as conn:
216             cursor = conn.cursor()
217             cursor.execute("""
218             INSERT INTO videos (
219                 video_id, sport, video_url, local_path, license_type,
220                 license_url, annotation_license, is_commercial, status, match_name, fps,
resolution
221                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
222             ON CONFLICT(video_id) DO UPDATE SET
223                 sport=excluded.sport,
224                 video_url=excluded.video_url,
225                 local_path=excluded.local_path,
226                 license_type=excluded.license_type,
227                 license_url=excluded.license_url,
228                 annotation_license=excluded.annotation_license,
229                 is_commercial=excluded.is_commercial,
230                 status=excluded.status,
231                 match_name=excluded.match_name,
232                 fps=excluded.fps,
233                 resolution=excluded.resolution
234             """, (
235                 video.video_id,
236                 video.sport.value,
237                 video.video_url,
238                 video.local_path,
239                 video.license_type.value,
240                 video.license_url,
241                 video.annotation_license.value if video.annotation_license else None,
242                 1 if video.is_commercial else 0,
243                 video.status.value,
244                 video.match_name,
245                 video.fps,
246                 video.resolution
247             ))
248             conn.commit()
249             return True
250
251     def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
252         if not rallies:

```

```

253         return 0
254     with self._get_connection() as conn:
255         cursor = conn.cursor()
256         # First clear existing rallies for this video_id to avoid key conflicts on
updates
257         video_id = rallies[0].video_id
258         cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
259
260         for rally in rallies:
261             cursor.execute("""
262                 INSERT INTO badminton_rallies (
263                     video_id, rally_number, start_time, end_time,
264                     score_before, score_after, shots_count, ending_reason,
last_shot_outcome
265                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
266                 """, (
267                     rally.video_id,
268                     rally.rally_number,
269                     rally.start_time,
270                     rally.end_time,
271                     rally.score_before,
272                     rally.score_after,
273                     rally.shots_count,
274                     rally.ending_reason,
275                     rally.last_shot_outcome
276                 ))
277         conn.commit()
278         return len(rallies)
279
280     def insert_tennis_points(self, points: List[TennisPoint]) -> int:
281         if not points:
282             return 0
283         with self._get_connection() as conn:
284             cursor = conn.cursor()
285             video_id = points[0].video_id
286             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
287
288             for pt in points:
289                 cursor.execute("""
290                     INSERT INTO tennis_points (
291                         video_id, point_number, start_time, end_time,
292                         set_score, game_score, server, ending_type
293                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
294                     """, (
295                         pt.video_id,
296                         pt.point_number,
297                         pt.start_time,
298                         pt.end_time,
299                         pt.set_score,
300                         pt.game_score,
301                         pt.server,
302                         pt.ending_type
303                     ))
304             conn.commit()
305             return len(points)
306
307     def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
308         if not deliveries:
309             return 0
310         with self._get_connection() as conn:
311             cursor = conn.cursor()
312             video_id = deliveries[0].video_id
313             cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))

```

```

314
315         for deliv in deliveries:
316             cursor.execute("""
317                 INSERT INTO cricket_deliveries (
318                     video_id, over_num, ball_num, start_time, end_time,
319                     batsman, bowler, runs, wicket, extras
320                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
321             """, (
322                 deliv.video_id,
323                 deliv.over,
324                 deliv.ball,
325                 deliv.start_time,
326                 deliv.end_time,
327                 deliv.batsman,
328                 deliv.bowler,
329                 deliv.runs,
330                 1 if deliv.wicket else 0,
331                 deliv.extras
332             ))
333         conn.commit()
334         return len(deliveries)
335
336     def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
337         if not rallies:
338             return 0
339         with self._get_connection() as conn:
340             cursor = conn.cursor()
341             video_id = rallies[0].video_id
342             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
343 (video_id,))
344
345         for rally in rallies:
346             cursor.execute("""
347                 INSERT INTO pickleball_rallies (
348                     video_id, rally_number, start_time, end_time,
349                     score_before, score_after, shots_count, ending_reason,
350                     last_shot_outcome
351                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
352             """, (
353                 rally.video_id,
354                 rally.rally_number,
355                 rally.start_time,
356                 rally.end_time,
357                 rally.score_before,
358                 rally.score_after,
359                 rally.shots_count,
360                 rally.ending_reason,
361                 rally.last_shot_outcome
362             ))
363         conn.commit()
364         return len(rallies)
365
366     def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
367         if not rallies:
368             return 0
369         with self._get_connection() as conn:
370             cursor = conn.cursor()
371             video_id = rallies[0].video_id
372             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
373 (video_id,))
374
375         for rally in rallies:
376             cursor.execute("""
377                 INSERT INTO tabletennis_rallies (
378                     video_id, rally_number, start_time, end_time,

```

```

376         score_before, score_after, shots_count, ending_reason,
last_shot_outcome
377     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
378     """ , (
379         rally.video_id,
380         rally.rally_number,
381         rally.start_time,
382         rally.end_time,
383         rally.score_before,
384         rally.score_after,
385         rally.shots_count,
386         rally.ending_reason,
387         rally.last_shot_outcome
388     ))
389     conn.commit()
390     return len(rallies)
391
392 @staticmethod
393 def _row_to_video(row) -> VideoMetadata:
394     """Rehydrate a DB row into VideoMetadata.
395
396     B5: enum construction (SportType/LicenseType/CurationStatus) used to raise a
397     bare ValueError on any value not in the enum ? a typo or a value from an older
398     schema made the whole query loop explode with a cryptic message and no clue
399     WHICH row was bad. Now: license values unknown to the enum map to UNKNOWN and
400     are logged loudly (OQ2 default: map + log, don't lose data); sport/status have
401     no safe default so they still raise but with the offending video_id in context.
402     """
403     keys = row.keys()
404     ann = row['annotation_license'] if 'annotation_license' in keys else None
405     video_id = row['video_id']
406
407     def _safe_license(value, field: str) -> LicenseType:
408         """Parse a license enum value; map unknown -> UNKNOWN with a warning."""
409         if not value:
410             return LicenseType.UNKNOWN
411         try:
412             return LicenseType(value)
413         except ValueError:
414             logger.warning(
415                 "video %r has unrecognized %s=%r; mapping to UNKNOWN",
416                 video_id, field, value)
417             return LicenseType.UNKNOWN
418
419     try:
420         sport = SportType(row['sport'])
421     except ValueError as e:
422         # No safe default for sport ? surface which row is bad before re-raising.
423         raise ValueError(
424             f"video {video_id!r} has unrecognized sport={row['sport']!r}; "
425             f"cannot rehydrate (fix the DB row or extend SportType)") from e
426
427     try:
428         status = CurationStatus(row['status'])
429     except ValueError as e:
430         raise ValueError(
431             f"video {video_id!r} has unrecognized status={row['status']!r}; "
432             f"cannot rehydrate (fix the DB row)") from e
433
434     return VideoMetadata(
435         video_id=video_id,
436         sport=sport,
437         video_url=row['video_url'],
438         local_path=row['local_path'],
439         license_type=_safe_license(row['license_type'], 'license_type'),

```



```

502         score_after=row['score_after'],
503         shots_count=row['shots_count'],
504         ending_reason=row['ending_reason'],
505         last_shot_outcome=row['last_shot_outcome']
506     ))
507     return rallies
508
509 def get_tennis_points(self, video_id: str) -> List[TennisPoint]:
510     points = []
511     with self._get_connection() as conn:
512         cursor = conn.cursor()
513         cursor.execute("""
514         SELECT * FROM tennis_points
515         WHERE video_id = ?
516         ORDER BY point_number
517         """, (video_id,))
518         for row in cursor.fetchall():
519             points.append(TennisPoint(
520                 video_id=row['video_id'],
521                 point_number=row['point_number'],
522                 start_time=row['start_time'],
523                 end_time=row['end_time'],
524                 set_score=row['set_score'],
525                 game_score=row['game_score'],
526                 server=row['server'],
527                 ending_type=row['ending_type']
528             ))
529     return points
530
531 def get_cricket_deliveries(self, video_id: str) -> List[CricketDelivery]:
532     deliveries = []
533     with self._get_connection() as conn:
534         cursor = conn.cursor()
535         cursor.execute("""
536         SELECT * FROM cricket_deliveries
537         WHERE video_id = ?
538         ORDER BY over_num, ball_num
539         """, (video_id,))
540         for row in cursor.fetchall():
541             deliveries.append(CricketDelivery(
542                 video_id=row['video_id'],
543                 over=row['over_num'],
544                 ball=row['ball_num'],
545                 start_time=row['start_time'],
546                 end_time=row['end_time'],
547                 batsman=row['batsman'],
548                 bowler=row['bowler'],
549                 runs=row['runs'],
550                 wicket=bool(row['wicket']),
551                 extras=row['extras']
552             ))
553     return deliveries
554
555 def get_pickleball_rallies(self, video_id: str) -> List[PickleballRally]:
556     rallies = []
557     with self._get_connection() as conn:
558         cursor = conn.cursor()
559         cursor.execute("""
560         SELECT * FROM pickleball_rallies
561         WHERE video_id = ?
562         ORDER BY rally_number
563         """, (video_id,))
564         for row in cursor.fetchall():
565             rallies.append(PickleballRally(
566                 video_id=row['video_id'],

```

```

567         rally_number=row['rally_number'],
568         start_time=row['start_time'],
569         end_time=row['end_time'],
570         score_before=row['score_before'],
571         score_after=row['score_after'],
572         shots_count=row['shots_count'],
573         ending_reason=row['ending_reason'],
574         last_shot_outcome=row['last_shot_outcome']
575     ))
576     return rallies
577
578 def get_tabletennis_rallies(self, video_id: str) -> List[TableTennisRally]:
579     rallies = []
580     with self._get_connection() as conn:
581         cursor = conn.cursor()
582         cursor.execute("""
583             SELECT * FROM tabletennis_rallies
584             WHERE video_id = ?
585             ORDER BY rally_number
586             """, (video_id,))
587         for row in cursor.fetchall():
588             rallies.append(TableTennisRally(
589                 video_id=row['video_id'],
590                 rally_number=row['rally_number'],
591                 start_time=row['start_time'],
592                 end_time=row['end_time'],
593                 score_before=row['score_before'],
594                 score_after=row['score_after'],
595                 shots_count=row['shots_count'],
596                 ending_reason=row['ending_reason'],
597                 last_shot_outcome=row['last_shot_outcome']
598             ))
599     return rallies
600
601 def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
602     """Return chronologically-ordered (start_time, end_time) tuples for a video,
603     regardless of sport. This is the sport-agnostic view used to export
604     indexer-ready evaluation/training labels.
605
606     Sports that have no annotation table yet (e.g. tabletennis, pickleball)
607     yield an empty list rather than raising.
608     """
609     mapping = SEGMENT_TABLES.get(sport)
610     if mapping is None:
611         return []
612     table, order_by = mapping # both come from the trusted map above, never user
input
613     with self._get_connection() as conn:
614         cursor = conn.cursor()
615         cursor.execute(
616             f"SELECT start_time, end_time FROM {table} "
617             f"WHERE video_id = ? ORDER BY {order_by}",
618             (video_id,))
619     )
620     return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]
621
622 def update_status(self, video_id: str, status: CurationStatus) -> bool:
623     with self._get_connection() as conn:
624         cursor = conn.cursor()
625         cursor.execute("UPDATE videos SET status = ? WHERE video_id = ?",
(status.value, video_id))
626         conn.commit()
627         return cursor.rowcount > 0
628
629 def delete_video(self, video_id: str) -> bool:

```



```

630         """Deletes a video and cascading child entries."""
631         with self._get_connection() as conn:
632             cursor = conn.cursor()
633             # Execute cascade deletes manually just in case SQLite foreign key support
is not active
634             cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
635             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
636             cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
637             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))
638             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))
639             cursor.execute("DELETE FROM videos WHERE video_id = ?", (video_id,))
640             conn.commit()
641             return cursor.rowcount > 0
642

```

12. user (2026-07-02T02:48:40.222Z)

```

1      # Next Steps ? Parked Work
2
3      > **Status:** `PARKED` ? these items are intentionally deferred with explicit "unpark"
triggers. They are NOT forgotten; they become active when the trigger fires.
4      > **Owner:** @avidullu . **Created:** 2026-06-24
5      > **Source:** parked from `docs/CODE_HEALTH_PLAN.md` $7 Tier D per decision D8.
6
7      These two structural items were parked during the 2026-06-24 code-health sweep because
they are either speculative (no current need) or messy to test thoroughly without the owner's
golden corpus. Each has a clear criticality trigger below.
8
9      ---
10
11     ## A1 ? Crawler retry/backoff convergence
12
13     **What:** The ShuttleSet crawler (`src/crawlers/badminton_crawler.py`) downloads per-set
CSVs and videos with raw `urllib.request.urlretrieve` ? **no retries, no backoff, no rate-
limiting**. The `fetch` CLI command already has proper retry logic (`_fetch_one_with_retry` in
`src/cli/curate.py`, backed by `download_best_video` in `src/core/downloader.py`). This item
converges the two download paths onto the shared, retried downloader.
14
15     **Why parked:** The crawler's batch semantics (per-match download limits, cumulative
size accounting, rollback-on-validation-failure) are genuinely different from `fetch`'s single-
video retry. A clean merge needs careful design **and testing against the live ShuttleSet repo**
(which can't be done offline ? it's golden-corpus / live-network territory for the owner). A
half-tested convergence refactor risks breaking the crawl's accounting invariants.
16
17     **Unpark trigger ? ANY of:**
18     - A real crawl run gets throttled and produces partial data (the pain becomes acute).
19     - The owner has a window to test against the live ShuttleSet repo.
20     - A second crawler (e.g. for a new dataset) is added, making the duplication cost
clearer.
21
22     **When unparked, the design questions to answer first:**
23     1. Does the shared `download_best_video` retry loop subsume the crawler's per-set CSV
fetches, or only the video downloads? (CSVs are small; the video path is where throttling
bites.)
24     2. How does retry interact with the cumulative `limit_max_total_size_mb` /
`limit_max_video_count` accounting? (A retried download shouldn't double-count.)
25     3. Should the crawler gain a configurable inter-match sleep (like `fetch`'s `--sleep`)?
26
27     **Starting point:** `src/core/downloader.py:download_best_video` +
`src/cli/curate.py:_fetch_one_with_retry` are the shared primitives. The crawler's
`fetch_and_curate` is the integration site.

```

```

28
29     ---
30
31     ## A3 ? Read-only DB connection path
32
33     **What:** `src/core/db.py` opens+closes a fresh SQLite connection per operation via
`_get_connection`. Read queries (`get_video`, `get_videos_by_status`, `get_segment_intervals`,
etc.) open implicit write transactions, which locks the DB. This is fine at single-user CLI
scale (concurrency = 1), but as soon as a second process or thread accesses the DB concurrently
(e.g. a web backend for khelsutra.guru serving while a crawl runs), readers will block writers
and vice versa.
34
35     **Why parked:** Pure speculative work until a backend exists. Today the collector is a
CLI run by one operator at a time; there is no concurrency. Building a read-only path now would
be untested-in-practice complexity.
36
37     **Unpark trigger ? when concurrency > 1:**
38     - A web backend (the khelsutra.guru API) starts reading from `sports_metadata.db` while
a crawl/import is running. **This is the critical moment** ? the first `database is locked`
error in production.
39     - Alternatively: the indexer (badminton-highlight-indexer) starts reading the
collector's DB directly rather than via the exported manifest.
40
41     **When unparked, the approach:**
42     1. Add a `_get_read_connection()` that sets `PRAGMA query_only = ON` (prevents
accidental writes) and uses `isolation_level=None` (autocommit, so reads don't hold a write
transaction).
43     2. Switch all `get_*` methods to use it; keep `_get_connection` for writers.
44     3. Consider WAL mode (`PRAGMA journal_mode = WAL`) for concurrent reader/writer support
? SQLite handles this well, but it's a one-way DB-format change that should be deliberate.
45     4. Consider a single long-lived connection with a thread lock instead of open/close per
call (matters only under load).
46
47     **Reference:** the indexer repo (`badminton-highlight-indexer`) uses a `PRAGMA
user_version` migration ladder and a connection-management pattern worth aligning with ? see
`backend/infrastructure/database.py` there.
48
49     ---
50
51     ## Relationship to the code-health plan
52
53     Both items are tracked in `docs/CODE_HEALTH_PLAN.md` $7 Tier D as ? NEXT_STEPS. The
plan's definition-of-done does NOT require them ? they are explicitly out of scope for the
current sweep. When either unpark trigger fires, lift the item back into an active work item (or
a fresh project doc) and update both docs.
54

```

13. user (2026-07-02T02:48:56.280Z)

```

1     import argparse
2     import sys
3     from src.core.config import load_config, get_database_path
4     from src.core.db import SportsDatabase
5     from src.parsers.badminton.coachai import CoachAIBadmintonParser
6
7     def main():
8         parser = argparse.ArgumentParser(description="Ingest external datasets into the
curation staging area")
9         parser.add_argument("--sport", required=True, choices=["badminton"], help="Sport
type")
10        parser.add_argument("--parser", required=True, choices=["coachai"], help="Parser
type to use")
11        parser.add_argument("--file-path", required=True, help="Path to raw dataset file")
12        parser.add_argument(
13            "--annotation-license", "--license", dest="annotation_license", default="MIT",

```

```

14         help="License of the ANNOTATIONS/dataset (e.g. ShuttleSet MIT). Recorded for "
15         "provenance only; does NOT make the video commercially usable.",
16     )
17     parser.add_argument(
18         "--video-license", dest="video_license", default="Unknown",
19         help="License of the VIDEO FOOTAGE ? this is what gates commercial use. "
20         "Broadcast/third-party video is 'Unknown' (default, non-commercial) unless
21         "you actually hold the footage rights.",
22     )
23     parser.add_argument("--video-url", default="https://youtube.com/watch?v=example",
24     help="URL to the video")
25     parser.add_argument("--match-name", help="Name of the match")
26     parser.add_argument("--video-id", help="Video unique identifier")
27
28     args = parser.parse_args()
29
30     # Load db path from the central config (falls back to the default).
31     # NOTE: this importer is intentionally badminton/CoachAI-only for now; other
32     # sports are populated via `curate import-local` or vendor golden sets.
33     db = SportsDatabase(get_database_path(load_config()))
34
35     if args.sport == "badminton" and args.parser == "coachai":
36         match_parser = CoachAIBadmintonParser()
37         try:
38             video_meta, rallies = match_parser.parse(
39                 args.file_path,
40                 video_id=args.video_id,
41                 video_url=args.video_url,
42                 annotation_license=args.annotation_license,
43                 video_license=args.video_license,
44                 match_name=args.match_name
45             )
46
47             # Insert into database (by default parsed entries are marked as staging)
48             db.insert_video(video_meta)
49             count = db.insert_badminton_rallies(rallies)
50
51             print(f"Successfully ingested match '{video_meta.match_name}' into
52             staging:")
53
54             print(f" - Video ID: {video_meta.video_id}")
55             print(f" - Total Rallies: {count}")
56             print(f" - Video license: {video_meta.license_type.value}")
57             print(f" - Annotation license: {video_meta.annotation_license.value if
58             video_meta.annotation_license else 'n/a'}")
59             print(f" - Commercial safe: {video_meta.is_commercial}")
60             if not video_meta.is_commercial:
61                 print(" ! NOT commercial-clean: this video is EXCLUDED from `curate
62                 export --commercial-only`.")
63             print("    Pass --video-license <held-rights-license> only if you
64             actually hold footage rights.")
65         except Exception as e:
66             print(f"Error parsing file: {e}")
67             sys.exit(1)
68
69     if __name__ == "__main__":
70         main()
71

```

14. user (2026-07-02T02:48:58.127Z)

```

1531 src/cli/curate.py
64 src/cli/ingest.py
249 src/cli/normalize_annotations.py
237 src/core/batch_preflight.py

```

```

30 src/core/config.py
641 src/core/db.py
195 src/core/downloader.py
61 src/core/manifests.py
77 src/core/parsing.py
366 src/core/proxy.py
149 src/core/validator.py
23 src/core/schemas/_mixins.py
14 src/core/schemas/badminton.py
89 src/core/schemas/base.py
15 src/core/schemas/cricket.py
14 src/core/schemas/pickleball.py
14 src/core/schemas/tabletennis.py
13 src/core/schemas/tennis.py
0 src/parsers/cvat/__init__.py
746 src/parsers/cvat/rally_cvat.py
193 src/parsers/badminton/coachai.py
333 src/crawlers/badminton_crawler.py
83 src/reporting/__init__.py
176 src/reporting/adapters.py
210 cleanup.py
5523 total
---tests---
tests/test_batch_preflight.py:14
tests/test_cleanup.py:5
tests/test_cli.py:28
tests/test_config.py:4
tests/test_crawler_tempfile.py:2
tests/test_cvat_parser.py:31
tests/test_cvat_robustness.py:7
tests/test_db.py:21
tests/test_db_pragmas.py:1
tests/test_downloader.py:14
tests/test_import_guard.py:5
tests/test_license_consistency.py:8
tests/test_licensing_gate.py:4
tests/test_local_import.py:5
tests/test_manifests.py:7
tests/test_normalize_annotations.py:8
tests/test_parser.py:6
tests/test_parsing.py:7
tests/test_proxy.py:44
tests/test_reporting.py:9
tests/test_reporting_faq.py:2
tests/test_reporting_noraise.py:5
tests/test_schemas.py:4
tests/test_validator.py:7

```

15. user (2026-07-02T02:49:10.233Z)

```

1 import os
2 import urllib.request
3 import urllib.parse
4 import csv
5 from typing import Dict, Any, List, Tuple
6 from tabulate import tabulate
7
8 from src.core.config import load_config
9 from src.core.db import SportsDatabase
10 from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
11 from src.core.schemas.badminton import BadmintonRally
12 from src.parsers.badminton.coachai import CoachAIBadmintonParser
13 from src.core.validator import DatasetValidator
14 from src.core.downloader import download_best_video
15

```

```

16     class ShuttleSetCrawler:
17         MATCH_CSV_URL = "https://raw.githubusercontent.com/wywyWang/CoachAI-
Projects/main/ShuttleSet/set/match.csv"
18         RAW_BASE_URL = "https://raw.githubusercontent.com/wywyWang/CoachAI-
Projects/main/ShuttleSet/set"
19
20     def __init__(self, config_path: str = "config/config.yaml"):
21         self.config_path = config_path
22         self.config = self._load_config()
23         self.db_path = self.config.get("database_path", "data/sports_metadata.db")
24         self.db = SportsDatabase(self.db_path)
25         self.validator = DatasetValidator(config_path)
26
27         # Load storage config
28         storage = self.config.get("video_storage", {})
29         self.local_dir = storage.get("local_dir", "./data/videos")
30         self.max_download_size_mb = storage.get("max_download_size_mb", 1000)
31         self.auto_download = storage.get("auto_download", True)
32         # Optional resolution cap (e.g. 1080 to avoid pulling 4K). Unset = best
available.
33         self.max_height = storage.get("max_height")
34
35         # Load ingestion limits
36         limits = self.config.get("ingestion_limits", {})
37         self.limit_max_total_size_mb = limits.get("max_total_download_size_mb", 5000)
38         self.limit_max_video_count = limits.get("max_total_video_count", 25)
39
40         os.makedirs(self.local_dir, exist_ok=True)
41         os.makedirs("data/temp", exist_ok=True)
42
43     def _load_config(self) -> Dict[str, Any]:
44         return load_config(self.config_path)
45
46     def fetch_and_curate(self):
47         """Downloads ShuttleSet metadata, crawls match videos, parses rallies, and
enforces limits."""
48         print("Starting ShuttleSet dataset ingestion...")
49         print(f"Ingestion limits: Max Videos={self.limit_max_video_count}, Max Total
Size={self.limit_max_size_mb_str()}")
50
51         # 1. Download match.csv metadata
52         temp_match_csv = "data/temp/shuttleaset_match.csv"
53         try:
54             print(f"Downloading match catalog from {self.MATCH_CSV_URL}...")
55             urllib.request.urlretrieve(self.MATCH_CSV_URL, temp_match_csv)
56         except Exception as e:
57             print(f"Error fetching match list: {e}")
58             return
59
60         # Read matches
61         matches: List[Dict[str, Any]] = []
62         with open(temp_match_csv, mode='r', encoding='utf-8') as f:
63             reader = csv.DictReader(f)
64             for row in reader:
65                 matches.append(row)
66
67         # 2. Iterate through matches
68         videos_downloaded = 0
69         total_size_downloaded_mb = 0.0
70         total_rallies_indexed = 0
71         db_videos_added = 0
72         db_videos_updated = 0
73
74         # Detailed summary logs
75         summary_log: List[Dict[str, Any]] = []

```

```

76
77         print(f"Found {len(matches)} matches in ShuttleSet catalog.")
78
79         for idx, match in enumerate(matches):
80             # Check limits
81             if videos_downloaded >= self.limit_max_video_count:
82                 print(f"\nLimit reached: Maximum video count
({self.limit_max_video_count}) reached. Stopping ingestion.")
83                 break
84             if total_size_downloaded_mb >= self.limit_max_total_size_mb:
85                 print(f"\nLimit reached: Maximum download size
({self.limit_max_total_size_mb} MB) reached. Stopping.")
86                 break
87
88             video_dir_name = match.get("video")
89             youtube_url = match.get("url")
90             match_id = match.get("id")
91
92             # Skip catalog rows missing the fields we depend on rather than
93             # crashing the entire ingestion run.
94             if not video_dir_name or not match_id:
95                 print(f"\n[{idx+1}/{len(matches)}] Skipping catalog row with missing
'video'/'id' column.")
96                 continue
97
98             try:
99                 set_count = int(float(str(match.get("set", "2")).strip() or "2"))
100             except ValueError:
101                 set_count = 2
102             match_name = video_dir_name.replace("_", " ")
103             video_id = f"shuttleset_{match_id}"
104
105             # ShuttleSet is MIT licensed -> commercially friendly
106             license_type = LicenseType.MIT
107             is_commercial = True
108
109             print(f"\n[{idx+1}/{len(matches)}] Processing: {match_name}")
110             print(f"  YouTube URL: {youtube_url}")
111
112             # Download video files (if configured)
113             local_path = None
114             download_size_mb = 0.0
115
116             probed_resolution = None
117             probed_fps = None
118             if self.auto_download and youtube_url:
119                 # 2.1 Download the best-resolution stream available (MP4). The shared
120                 # downloader sorts formats by resolution and records the actual
121                 # width/height/fps it obtained ? we never assume a resolution.
122                 print("  Downloading best-resolution video...")
123                 video_filename = f"{video_id}.mp4"
124                 dest_path = os.path.join(self.local_dir, video_filename)
125
126                 result = download_best_video(
127                     youtube_url, dest_path,
128                     max_height=self.max_height,
129                     max_size_mb=self.max_download_size_mb,
130                 )
131                 if result.success:
132                     local_path = result.path
133                     download_size_mb = result.size_mb
134                     probed_resolution = result.resolution_label
135                     probed_fps = result.fps
136                     videos_downloaded += 1
137                     total_size_downloaded_mb += download_size_mb

```

```

138         print(f" Successfully downloaded video "
139               f"({download_size_mb:.2f} MB, {probed_resolution or 'unknown
res'})"
140               f"({f', {probed_fps:g}fps' if probed_fps else ''})")
141     else:
142         print(f" Failed to download video: {result.error}")
143         local_path = None
144         download_size_mb = 0.0
145
146     # 2.2 Download & Merge stroke CSV files for this match
147     print(" Fetching annotation files...")
148     rallies_merged: List[BadmintonRally] = []
149
150     # Temporary storage to merge set csvs
151     merged_csv_path = f"data/temp/{video_id}_merged.csv"
152     headers = ["set", "rally", "ball_round", "time", "type", "score_a",
"score_b"]
153
154     with open(merged_csv_path, 'w', newline='', encoding='utf-8') as mf:
155         writer = csv.writer(mf)
156         writer.writerow(headers)
157
158         for s in range(1, set_count + 1):
159             encoded_dir = urllib.parse.quote(video_dir_name)
160             set_url = f"{self.RAW_BASE_URL}/{encoded_dir}/set{s}.csv"
161             set_temp_path = f"data/temp/{video_id}_set{s}.csv"
162             # B6: cleanup is guaranteed in finally so a row-write or parse
163             # failure mid-set can't orphan the per-set CSV in data/temp.
164             try:
165                 urllib.request.urlretrieve(set_url, set_temp_path)
166
167                 # Read and write rows, adding set number
168                 with open(set_temp_path, 'r', encoding='utf-8') as sf:
169                     sreader = csv.DictReader(sf)
170                     # Normalize fieldnames
171                     sreader.fieldnames = [n.lower().strip() for n in
sreader.fieldnames] if sreader.fieldnames else []
172
173                     for row in sreader:
174                         # Write to merged file
175                         writer.writerow([
176                             s,
177                             row.get("rally", "0"),
178                             row.get("ball_round", row.get("ball_seq", "1")),
179                             row.get("time", row.get("timestamp", "0.0")),
180                             row.get("type", ""),
181                             row.get("score_a", row.get("round_score_a", "0")),
182                             row.get("score_b", row.get("round_score_b", "0"))
183                         ])
184             except Exception as e:
185                 print(f" Warning: Failed to fetch Set {s} CSV: {e}")
186             finally:
187                 # Always remove the per-set temp file, even on failure.
188                 if os.path.exists(set_temp_path):
189                     try:
190                         os.remove(set_temp_path)
191                     except OSError:
192                         pass
193
194     # 2.3 Parse merged annotations
195     rallies = []
196     video_meta = None
197     if os.path.exists(merged_csv_path):
198         parser = CoachAIBadmintonParser(start_buffer=1.0, end_buffer=2.0)
199         # B6: cleanup is guaranteed in finally so a parse failure can't orphan

```

```

200         # the merged CSV in data/temp.
201     try:
202         video_meta, rallies = parser.parse(
203             merged_csv_path,
204             video_id=video_id,
205             video_url=youtube_url,
206             license=license_type.value,
207             match_name=match_name,
208             local_path=local_path
209         )
210     except Exception as e:
211         print(f" Error parsing merged CSV: {e}")
212         rallies = []
213     finally:
214         if os.path.exists(merged_csv_path):
215             try:
216                 os.remove(merged_csv_path)
217             except OSError:
218                 pass
219
220     # Override the parser's placeholder resolution/fps with what we actually
221     # downloaded, so the DB reflects the real file (the indexer validates
222     # resolution and keys by it).
223     if video_meta is not None:
224         if probed_resolution:
225             video_meta.resolution = probed_resolution
226         if probed_fps:
227             video_meta.fps = probed_fps
228
229     if not video_meta or not rallies:
230         print(" Skipping match due to missing metadata or rallies.")
231         if local_path and os.path.exists(local_path):
232             os.remove(local_path)
233             # Roll back download accounting for the now-discarded file so
234             # the limit checks and summary stay accurate.
235             videos_downloaded -= 1
236             total_size_downloaded_mb -= download_size_mb
237         continue
238
239     # 2.4 Validate rallies
240     is_valid, errors = self.validator.validate_badminton_rallies(rallies)
241     if not is_valid:
242         print(f" Warning: Rallies failed validation. Skipping DB insertion.")
243         for err in errors:
244             print(f" - {err}")
245         if local_path and os.path.exists(local_path):
246             os.remove(local_path)
247             # Roll back download accounting for the now-discarded file.
248             videos_downloaded -= 1
249             total_size_downloaded_mb -= download_size_mb
250         continue
251
252     # 2.5 Insert to Database
253     # Update metrics
254     existing = self.db.get_video(video_id)
255     if existing:
256         db_videos_updated += 1
257     else:
258         db_videos_added += 1
259
260     self.db.insert_video(video_meta)
261     count = self.db.insert_badminton_rallies(rallies)
262     total_rallies_indexed += count
263
264     print(f" Successfully indexed {count} rallies in SQLite database.")

```



```

265
266         summary_log.append({
267             "match_name": match_name,
268             "video_id": video_id,
269             "downloaded": local_path is not None,
270             "size_mb": download_size_mb,
271             "rallies": count
272         })
273
274     # 3. Print execution summary
275     self._print_execution_summary(
276         videos_downloaded,
277         total_size_downloaded_mb,
278         total_rallies_indexed,
279         db_videos_added,
280         db_videos_updated,
281         summary_log
282     )
283
284     # Cleanup temp directory (best-effort).
285     try:
286         for f in os.listdir("data/temp"):
287             os.remove(os.path.join("data/temp", f))
288         os.rmdir("data/temp")
289     except OSError:
290         pass
291
292     def limit_max_size_mb_str(self) -> str:
293         return f"{self.limit_max_total_size_mb} MB"
294
295     def _print_execution_summary(self,
296                                videos_downloaded: int,
297                                total_size_mb: float,
298                                total_rallies: int,
299                                added: int,
300                                updated: int,
301                                log: List[Dict[str, Any]]):
302         print("\n" + "="*50)
303         print("                INGESTION EXECUTION SUMMARY                ")
304         print("="*50)
305         print(f"Total Video Files Downloaded: {videos_downloaded}")
306         print(f"Total Download Size           : {total_size_mb:.2f} MB")
307         print(f"Total Rallies Indexed           : {total_rallies}")
308         print(f"Database Matches Added          : {added}")
309         print(f"Database Matches Updated       : {updated}")
310         print("-"*50)
311
312         if log:
313             print("Match Log Breakdown:")
314             table_data = []
315             for item in log:
316                 table_data.append([
317                     item["match_name"],
318                     item["video_id"],
319                     "Yes" if item["downloaded"] else "No",
320                     f"{item['size_mb']:.2f} MB",
321                     item["rallies"]
322                 ])
323             print(tabulate(table_data, headers=["Match Name", "Video ID", "Downloaded",
324 "File Size", "Rallies"], tablefmt="simple"))
325         else:
326             print("No matches were successfully processed.")
327         print("="*50 + "\n")
328
329     def main():

```

```

329         crawler = ShuttleSetCrawler()
330         crawler.fetch_and_curate()
331
332     if __name__ == "__main__":
333         main()
334

```

16. user (2026-07-02T02:49:24.509Z)

Decision records

Lightweight ADRs for this repo. (The cross-repo training-pipeline ADRs live in the indexer repo's `docs/DECISIONS.md`; this file is for collector-local choices.)

ADR-001 ? Keep explicit per-sport code instead of a sport registry (for now)

Status: Accepted . **Date:** 2026-06-02

Context

Supporting a sport currently requires parallel edits across several sites:

- `SEGMENT\_TABLES` + a `CREATE TABLE` + `insert\_` / `get\_` in `src/core/db.py`
- a `validate\_` method in `src/core/validator.py`
- `import-local` parse/validate/insert branches and interactive review/play in `src/cli/curate.py`
- a schema in `src/core/schemas/`

A code review flagged this as copy-paste duplication and proposed a registry/plugin mechanism (e.g. `SPORT\_REGISTRY: dict[SportType, SportsSpec]` driving generic insert/get/validate/dispatch) to make adding a sport a single entry.

Decision

We are keeping the explicit per-sport code and NOT introducing a registry abstraction at this time.

Rationale

- **It's well-tested and mechanical.** Coverage is ~87%; adding a sport is a known, ~30-minute "mirror an existing sport" task (checklist below), not a research problem.
- **Schemas are heterogeneous.** Rally sports (badminton, pickleball, table tennis) share `rally\_number` / `shots\_count` / `ending\_reason`, but tennis (`set\_score`, `game\_score`, `server`, `ending\_type`) and cricket (`over`, `ball`, `runs`, `wicket`, `extras`) differ enough that a single generic table/row mapper would leak special-cases ? trading visible duplication for hidden complexity.
- **Explicit code is greppable.** Contributors can follow one sport end-to-end without learning a registry indirection. Premature abstraction is harder to undo than dup.
- **Low, slow growth.** Five sports today; new ones arrive rarely.

Consequences

- Adding a sport touches ~5 sites (see checklist). We accept that boilerplate.
- If the cost rises, ADR-001 should be revisited and superseded.

Revisit triggers (when to build the registry)

- Sport count grows beyond ~8, **or**
- We add a 4th+ rally-identical sport and the dispatch churn starts causing bugs, **or**
- A new cross-cutting feature would otherwise require editing every per-sport branch.

Future design sketch (when revisited)

```

...
@dataclass
class SportsSpec:
    schema_class: type

```

```
table_name: str
pk_columns: tuple[str, ...]
columns: tuple[str, ...]
validate_fn: Callable
```

```
SPORT_REGISTRY: dict[SportType, SportSpec] = { ... }
```

Generic: insert_segments(sport, items) / get_segments(sport, video_id) / validate(sport, items) / import-local dispatch driven by the registry.

```
===archives===
```

```
docs/annotation:
```

```
ANNOTATION_GUIDE.md
```

```
INTAKE_MANIFEST_TEMPLATE.csv
```

```
NDA_TEMPLATE.md
```

```
ROORA_BRIEF.md
```

```
ROORA_RUNBOOK.md
```

```
docs/archives:
```

```
CODE_HEALTH_PLAN.md
```

```
README.md:11:- **SQLite Backend**: Centralized `sports_metadata.db` with a generic `videos` table and sport-specific annotation tables (`badminton_rallies`, `tennis_points`, `cricket_deliveries`, `pickleball_rallies`, `tabletennis_rallies`).
```

```
docs/NEXT_STEPS.md:38:- A web backend (the khelsutra.guru API) starts reading from `sports_metadata.db` while a crawl/import is running. **This is the critical moment** ? the first `database is locked` error in production.
```

17. user (2026-07-02T02:49:25.609Z)

```
136:python -m pytest tests/ --cov=src --cov-report=term-missing # with coverage (~87%)
```

```
===readme headings===
```

```
1:# Sports Data Collector & Curator
```

```
9:## Features
```

```
20:## Installation
```

```
32:### External requirements (not pip-installable)
```

```
47:## Usage
```

```
52:# On Windows PowerShell:
```

```
56:### 1. View Database Status
```

```
62:### 2. Ingest External Datasets to Staging
```

```
68:### 3. Launch the Interactive Curation Tool & Play Harness
```

```
74:### 4. Manually Import Local Videos & Annotations
```

```
81:### 5. Fetch / Upgrade Videos to Best Resolution
```

```
84:# Upgrade every sub-720p video to best resolution, using a logged-in browser's cookies:
```

```
87:# A single video, forcing a re-download even if a good local copy exists:
```

```
96:### 6. Export Annotations as Indexer-Ready Eval/Training Sets
```

```
99:# Curated + commercially-safe videos across all supported sports (default):
```

```
102:# One sport, also including staging entries:
```

```
109:## Configuration
```

```
119:## Documentation
```

```
131:## Running Tests
```

18. user (2026-07-02T02:49:40.842Z)

```
8:## ADR-001 ? Keep explicit per-sport code instead of a sport registry (for now)
```

```
76:## ADR-002 ? Annotation proxies are generated in-repo (`curate prep-annotation-proxy`), not by a one-off tool
```

```
===
```

```
===indexer ADR-008===
```

19. user (2026-07-02T02:49:42.940Z)

```
src/cli/curate.py:135:         except Exception:
src/cli/curate.py:203:         except Exception as e:
src/cli/curate.py:836:         except BaseException:
src/cli/ingest.py:59:         except Exception as e:
```

```

src/core/batch_preflight.py:175:         except Exception as ex: # ProxyError or anything probe
raises
src/crawlers/badminton_crawler.py:56:         except Exception as e:
src/crawlers/badminton_crawler.py:184:         except Exception as e:
src/crawlers/badminton_crawler.py:210:         except Exception as e:
src/reporting/__init__.py:20:except Exception: # pragma: no cover - only when obsreport is
absent
src/reporting/__init__.py:48:     except Exception: # reporting must never break the command
src/reporting/__init__.py:81:     except Exception: # reporting must never break the command
===import_annotations===
44:     maturity gate: ``staging`` = needs review, ``curated`` = confirmed)."""
145:     def import_annotations(
631:         from src.core.proxy import md5_file # local import (mirrors prep_annotation_proxy)
679:         md5 = None
681:         md5 = md5_file(video.local_path)
684:         f"exporting without an md5 (the indexer join needs the exact
file).")
690:         "md5": md5,
804:         source_md5 = proxylib.md5_file(src)
807:         if r.get("video_id") == video_id and r.get("source_md5") != source_md5]
810:         f"'{video_id}' to a DIFFERENT source (md5
{clashes[-1].get('source_md5')})).")
814:         if any(r.get("video_id") == video_id and r.get("source_md5") == source_md5
850:         proxy_md5 = proxylib.md5_file(out)
856:         "source_md5": source_md5,
858:         "proxy_md5": proxy_md5,
887:         print(f" Source      : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5
{source_md5})")
888:         print(f" Proxy      : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5
{proxy_md5})")

```

20. user (2026-07-02T02:49:58.492Z)

```

./badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md:388:## ADR-008: Owned-model
topology ? artifact boundary now, repo split at Gen-2
./badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md:498: failure mode is
documented in ADR-008's rationale) with pointer READMEs in `kshelsutra`.
./badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md:691: `ADR-005` (permissive
licensing ? engine + image stay MIT/Apache/BSD) . `ADR-008` (train==serve featurizer single-
sourcing ? the parity discipline this extends to local?cloud) . `ADR-009` (**local-first
delivery** ? origin of the truly-local profile) . `ADR-010` (DECOUPLED DO?GCP) . `ADR-011`
(**deferred M3 serving + M4 billing** ; override "rent nothing / not a service"; $06 gates =
DPA/consent, collector `md5`, WASB-C3) . `ADR-012` (**Cloud Run+GPU = serving model** ; NVDEC
lever) . `ADR-013` (**GCP sole compute stack**) . **? ADR-014** (this). Non-ADR: realizes
`PLATFORM_ARCHITECTURE.md`'s `ComputeBackend` / `StorageBackend` registries; the truly-local
profile = `VIDEO_LOCALITY_MODEL.md` L0 + `DESKTOP_APP_PLAN.md` `LocalDesktop`; informed by the
archived `DEPLOY_REPORT_GCP_2026-06-20` (cross-GPU sub-pixel parity finding).
./badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/01-EXISTING-PLANS-
AND-GAP.md:61: **ADR-008 train/serve wall**, where "the boundary ? is an artifact bundle
(weights + manifest), never a
./badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-
CONTRACT.md:59:- **Train/serve wall (ADR-008):** the boundary between training and serving is an
**artifact bundle**
./badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-
PLAN.md:44:5. Upload weights + manifest to `weights/<semver>/` (ADR-008 artifact bundle,
content/semver-keyed).
./badminton-highlight-indexer/docs/CODE_MAP.md:31:| Training code (model fitting) | `training/`
| Behind the CI-enforced import wall (ADR-008) ? `backend.main` must not import it. Own
`requirements-train.txt`. |
./badminton-highlight-indexer/docs/CODE_MAP.md:35:| A model artifact (weights + manifest) |
`artifacts/<name>/<ver>/` | ADR-008: **manifest committed** (provenance), **weights
gitignored**. |
./badminton-highlight-indexer/docs/CODE_MAP.md:177:- `@backend/features/rally_seq.py` ? **THE
single source of truth for the Gen-0 rally_seq per-frame feature schema** (ADR-008): training
(`training/`) AND the serving/eval path import the SAME `build_frame_arrays` / `features` so

```

```

train==serve by construction. Pure numpy/scipy (no torch/cv2). `::FEATURE_SCHEMA_VERSION` (=1;
bump on ANY semantic change ? artifacts pin it, loader hard-fails on mismatch), `::FEAT_NAMES`
(the 6: `vis_rate,spd_mean,spd_max,cross_rate,x_std,y_std`), `::FEATURE_SCHEMA`
(`win_sec`,`stride_sec`,`fps_semantics`,`occlusion_rule`),
`::build_frame_arrays(points,frame_width,fps)` (per-frame vis/x/y + per-visible-frame speed &
net-crossings; speed normalized per-SECOND `*fps`, NOT per-frame ? the 2026-06-10 fps-skew fix;
skips speed/crossings across occlusion gaps > `max(2,round(0.25*fps))`),
`::features(arr,fps,win_sec,stride_sec)` (scipy `uniform/maximum_filter1d` windowed -> `(X[n,6],
times)`). ? re-exported by `@backend/eval/rally_seq_proto.py`.
./badminton-highlight-indexer/docs/CODE_MAP.md:178:- `@backend/eval/rally_seq_proto.py` ?
**PROTOTYPE** learned per-FRAME rally segmenter = owned-model **Gen-0 seed** (PR #61). 6
features over the WASB trajectory (`vis_rate, spd_mean, spd_max, cross_rate, x_std, y_std`) ->
`classifier.LogisticRegression`, honest LOVO. ? `build_frame_arrays`/`features` (+`FEAT_NAMES`)
are NOT defined here ? they are **re-exported** (`noqa F401`) from
`@backend/features/rally_seq.py` (ADR-008 single-sourcing; speed per-SECOND + occlusion-gap-
guarded since the 2026-06-10 sweep). Locally-owned: `::labels_for`, `::probs_to_segments`
(smooth->threshold->merge/min-dur), `::roc_auc` (rank-based with **midrank ties**,
scipy.stats.rankdata), `::run` (per-video AUC + LOVO-threshold F1 + oracle-threshold F1). ?
DIRECTIONAL only (n=6, linear, threshold-transfer unsolved); held-out per-frame AUC 0.83?0.92,
**beats the heuristic on multi-court footage** (Kushagra/gBaaddy). **n=6 LOVO: learned 0.626 vs
heuristic 0.480** (learned rose from 0.583 after the fps-normalization fix). Manifest
`output/human_lovo_manifest.json` (n=6; trajectory paths now under `Annotation
Setup/Collect/Trajectories/`) shared w/ calibrate_local.
===indexer decisions ADR list===
grep: badminton-highlight-indexer/docs/DECISIONS.md: No such file or directory

```

21. user (2026-07-02T02:50:11.224Z)

```

100         def _annotation_key(sport_str: str, a: Annotation) -> Union[int, Tuple[int, int],
None]:
101             """The per-sport primary-key value of one annotation (for duplicate
detection)."""
102             if sport_str in ("badminton", "pickleball", "tabletennis"):
103                 return getattr(a, "rally_number", None)
104             if sport_str == "tennis":
105                 return getattr(a, "point_number", None)
106             if sport_str == "cricket":
107                 return (getattr(a, "over", None), getattr(a, "ball", None))
108             return None
109
110         def _guard_import(self, sport_str: str, video_id: str, annotations:
List[Annotation],
111                             force: bool = False) -> None:
112             """Pre-insert safety checks (run BEFORE any DB write, so a rejection is
atomic)."""
113             # (1) Duplicate primary-key guard. int(float()) truncation (e.g. 7 and 7.5 both
114             # become 7) would otherwise raise IntegrityError mid-insert, leaving a committed
115             # video row with rolled-back annotations. Catch it before touching the DB.
116             keys = [self._annotation_key(sport_str, a) for a in annotations]
117             keys = [k for k in keys if k is not None]
118             seen, dupes = set(), set()
119             for k in keys:
120                 if k in seen:
121                     dupes.add(k)
122                 seen.add(k)
123             if dupes:
124                 raise CurationError(
125                     f"Duplicate rally/point identifier(s) {sorted(dupes)} in the annotations
"
126                     f"(often a fractional id like 7.5 truncating to 7). "
127                     f"Re-run with --normalize, or normalize first: "
128                     f"python -m src.cli.normalize_annotations --in <csv> --out clean.csv")
129
130             # (2) Golden-label protection ? refuse to silently clobber an existing video's
131             # annotations (filename-derived video_ids collide easily).

```

```

132         if self.db.get_video(video_id):
133             try:
134                 n = len(self.db.get_segment_intervals(video_id, SportType(sport_str)))
135             except Exception:
136                 n = 0
137             if n > 0 and not force:
138                 raise CurationError(
139                     f"video_id '{video_id}' already has {n} annotated segment(s); "
140                     f"importing would REPLACE them. "
141                     f"Pass --force to overwrite, or use a distinct --video-id to keep
both.")
142             if n > 0:
143                 print(f"Warning: --force given ? replacing {n} existing segment(s) for
video_id '{video_id}'.")
144
145     def import_annotations(
146         self,
147         *,
148         sport: str,
149         video_path: Optional[str],
150         annotations_path: str,
151         video_id: Optional[str] = None,
152         match_name: Optional[str] = None,
153         license: str = "Proprietary",
154         fps: Optional[float] = None,
155         resolution: Optional[str] = None,
156         normalize: bool = False,
157         force: bool = False,
158         status: CurationStatus = CurationStatus.CURATED,
159     ) -> ImportResult:
160         """Ingest one video's rally labels into the DB and RETURN a structured result ?
the callable
161         core behind the CLI ``import-local`` (and a future UI/service contribute path).
Raises
162         ``CurationError`` on any user-correctable failure (unsupported sport,
missing/bad annotations,
163         validation, duplicate/clobber guard); it prints nothing and never calls
``sys.exit``, so a
164         service can catch and surface it.
165
166         ``status`` is the MATURITY GATE: pass ``CurationStatus.STAGING`` for a
contributed label that
167         NEEDS REVIEW (the safe default for programmatic/UI imports ? never auto-promote
to golden),
168         ``CURATED`` once confirmed. The CLI passes CURATED (a manual owner import is
reviewer-vetted).
169         """
170         warnings: List[str] = []
171         sport_str = (sport or "").lower()
172         if sport_str not in self.config.get("supported_sports", ["badminton"]):
173             raise CurationError(
174                 f"Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
175
176         if video_path and not os.path.exists(video_path):
177             warnings.append(f"Local video path '{video_path}' does not exist on disk,
but continuing registration.")
178         if not annotations_path or not os.path.exists(annotations_path):
179             raise CurationError(f"Annotations file '{annotations_path}' not found.")
180
181         match_name = match_name or os.path.basename(video_path or
annotations_path).split('.')[0]
182         vid = video_id or match_name.lower().replace(" ", "_")
183
184         try:

```

```

185         license_type = LicenseType(license)
186     except ValueError:
187         warnings.append(f"Unknown license '{license}'. Defaulting to Unknown (non-
commercial).")
188         license_type = LicenseType.UNKNOWN
189     is_commercial = self.validator.check_license_commercial(license_type)
190
191     # Optional: normalize a raw CSV annotator export before parsing (drops unusable
rows,
192     # rennumbers fractional/duplicate rally ids) so the guards below can't fire.
193     parse_path = annotations_path
194     if normalize and parse_path.lower().endswith(".csv"):
195         from src.cli.normalize_annotations import normalize_file
196         normalized_path = parse_path + ".normalized.csv"
197         rep = normalize_file(parse_path, normalized_path)
198         warnings.append("normalize-annotations:\n" + rep.summary())
199         parse_path = normalized_path
200
201     try:
202         annotations = self._parse_local_annotations(sport_str, vid, parse_path)
203     except Exception as e:
204         raise CurationError(f"Error parsing annotations: {e}") from e
205
206     is_valid, errors = True, []
207     if sport_str == "badminton":
208         is_valid, errors = self.validator.validate_badminton_rallies(annotations)
209     elif sport_str == "tennis":
210         is_valid, errors = self.validator.validate_tennis_points(annotations)
211     elif sport_str == "cricket":
212         is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
213     elif sport_str == "pickleball":
214         is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
215     elif sport_str == "tabletennis":
216         is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
217     if not is_valid:
218         raise CurationError("Annotations failed validation:\n" + "\n".join(f" -
{e}" for e in errors))
219
220     # Pre-insert safety (raises CurationError) ? atomic: no half-written DB on
rejection.
221     self._guard_import(sport_str, vid, annotations, force=force)
222
223     video_meta = VideoMetadata(
224         video_id=vid, sport=SportType(sport_str), video_url=None,
local_path=video_path,
225         license_type=license_type, license_url=None, is_commercial=is_commercial,
226         status=status, match_name=match_name, fps=fps, resolution=resolution,
227     )
228     self.db.insert_video(video_meta)
229     count = 0
230     if sport_str == "badminton":
231         count = self.db.insert_badminton_rallies(annotations)
232     elif sport_str == "tennis":
233         count = self.db.insert_tennis_points(annotations)
234     elif sport_str == "cricket":
235         count = self.db.insert_cricket_deliveries(annotations)
236     elif sport_str == "pickleball":
237         count = self.db.insert_pickleball_rallies(annotations)
238     elif sport_str == "tabletennis":
239         count = self.db.insert_tabletennis_rallies(annotations)
240
241     return ImportResult(
242         video_id=vid, count=count, status=status.value, match_name=match_name,
243         is_commercial=is_commercial, license_type=license_type.value,
244         warnings=warnings, video_meta=video_meta, annotations=annotations,

```

```

245         )
246
247     def import_local(self, args):
248         """CLI: manually register a local video + annotations (a curated owner import).
Thin wrapper
249         over ``import_annotations`` ? prints the warnings/summary, emits the
observability report, and
250         translates a ``CurationError`` into ``sys.exit(1)``."""
251         try:
252             res = self.import_annotations(
253                 sport=args.sport, video_path=args.video_path,
annotations_path=args.annotations_path,
254                 video_id=args.video_id, match_name=args.match_name,
license=args.license,
255                 fps=args.fps, resolution=args.resolution,
256                 normalize=getattr(args, "normalize", False), force=getattr(args,
"force", False),
257                 status=CurationStatus.CURATED, # a manual owner import is already
reviewer-vetted
258             )
259         except CurationError as e:
260             print(f"Error: {e}")
261             sys.exit(1)
262
263         for w in res.warnings:
264             print(f"Warning: {w}")
265         print(f"\nSuccessfully imported local video '{res.match_name}':")
266         print(f"  - Video ID: {res.video_id}")
267         print(f"  - Sport: {args.sport.lower()}")
268         print(f"  - Commercial Use Allowed: {res.is_commercial} (License:
{res.license_type})")
269         print(f"  - Registered {res.count} annotated segments.\n")
270
271         # Per-video observability report (next to the video). Never raises.
272         from src.reporting import emit_import_report
273         src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
274         report_paths = emit_import_report(
275             video_meta_obj=res.video_meta, annotations=res.annotations, is_valid=True,
errors=[], count=res.count, sport=args.sport.lower(), source_format=src_fmt,
276             fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
277         )
278     )
279     if report_paths:

```

22. user (2026-07-02T02:50:28.439Z)

```

e4dd8f1 2026-06-10 chore(corpus): reclassify 23 ShuttleSet broadcast videos as non-commercial
(owner-approved) (#17)
038424d 2026-06-07 feat(reporting): per-video observability report (collector)
===backup ignore line committed?===
*.report.json
*.report.md
*.summary.md

```

Local DB backups (e.g. data/sports_metadata.backup-YYYY-MM-DD.db) ? never comm

```

data/*.backup-*.db
===cleanup.py what is it===
#!/usr/bin/env python3
import os
import sys
import shutil
import argparse
import math
from typing import List

```



```

from src.core.config import load_config

def get_file_or_dir_size(path: str) -> int:
    """Calculates the size of a file or directory in bytes."""
    if not os.path.exists(path):
        return 0
    if os.path.isfile(path):
        return os.path.getsize(path)

    total_size = 0
    for dirpath, _, filenames in os.walk(path):
        for f in filenames:
            fp = os.path.join(dirpath, f)
            if os.path.exists(fp) and not os.path.islink(fp):
                total_size += os.path.getsize(fp)
    return total_size

def format_size(size_bytes: int) -> str:
    """Formats bytes to a human-readable string."""
    if size_bytes == 0:
        return "0 B"
    size_name = ("B", "KB", "MB", "GB")
    i = int(math.floor(math.log(size_bytes, 1024)))
    p = math.pow(1024, i)
    s = round(size_bytes / p, 2)
    return f"{s} {size_name[i]}"

def get_all_items_in_dir(path: str) -> List[str]:
    """Returns a list of absolute paths of all files/directories inside a directory."""
    items: List[str] = []
    if not os.path.exists(path) or not os.path.isdir(path):
        return items

```

23. user (2026-07-02T02:50:39.375Z)

```

51-
52:# Local generated artifacts (review reels, eval CSVs, e2e/proof outputs) -- regenerable,
never committed
53:artifacts/
54-
55-# OS generated files
56-.DS_Store
===indexer artifacts dir===
appliance_design.json
branch-prune-backup-24178a7.txt
cloudrun_e2e_validation
cloudrun_lightup
landing
long_rally_lens_n15.json
long_rally_lens_n15.txt
multishuttle_142
pr245_code_review.md
pr245_fixes_plan.md
pr245_review.json
pr245_review.md
promo
rally_tal_gen0
review
site_deploy
so.json
so.py
tridet.pdf
tridet.txt

```

24. user (2026-07-02T02:50:40.284Z)

```

        print("No conflated rows found ? every remote video already has a video-footage
license.")
        return

    print(f'{"APPLYING" if args.apply else "DRY-RUN"} ? {len(changed)} conflated row(s):')
    for v in changed:
        print(f" {v.video_id:40s} {v.license_type.value} (video) -> annotation_license; "
              f"video license -> Unknown; is_commercial {v.is_commercial} -> False")
        if args.apply:
            v.annotation_license = v.annotation_license or v.license_type
            v.license_type = LicenseType.UNKNOWN
            v.is_commercial = self.validator.check_license_commercial(v.license_type)
            self.db.insert_video(v)

    if args.apply:
        print("\nApplied. Re-run `curate export` to regenerate the manifest without these
videos.")
    else:
        print("\nDry-run only. Re-run with --apply to persist, then `curate export`.")

def export_eval_set(self, args):
    """Export curated annotations as indexer-ready eval/training sets.

    Emits, per qualifying video, a `

```

```

        if not intervals:
            skipped_empty += 1
            continue

        sport_dir = os.path.join(out_dir, sport.value)
        os.makedirs(sport_dir, exist_ok=True)
        csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
        with open(csv_path, "w", newline="", encoding="utf-8") as f:
            writer = csv.writer(f)
            writer.writerow(["start", "end"])
            for start, end in intervals:
                writer.writerow([start, end])

        # Full-file MD5 = the indexer's join key (it keys videos by file MD5, not our
video_id).
        # Computed when the local file is present; None (with a warning) for a URL-
only/offline
        # entry, so the downstream run knows the deterministic join isn't available for
it.

        md5 = None
        if video.local_path and os.path.exists(video.local_path):
            md5 = md5_file(video.local_path)
        elif video.local_path:
            print(f"Warning: '{video.video_id}' local file missing ({video.local_path});
"
                f"exporting without an md5 (the indexer join needs the exact file).")

        manifest.append({
            "video_id": video.video_id,
            "sport": sport.value,
            "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
            "md5": md5,
            "local_path": video.local_path,
            "video_url": video.video_url,
            "match_name": video.match_name,
            "fps": video.fps,
            "resolution": video.resolution,
            "license_type": video.license_type.value,
            "is_commercial": video.is_commercial,
            "status": video.status.value,
            "num_segments": len(intervals),
        })

```

25. user (2026-07-02T02:51:02.352Z)

docs/DECISIONS.md:130: precedent for the failure mode: unfilled intake-manifest TODOs, eval artifacts lost in

===code health plan status===

Sports Data Collector ? Code Health Plan

```

> Status: `DONE` ? `Tiers S+A+B+C complete; Tier D: A2/A5/A6 merged, A1/A3 parked in
NEXT_STEPS, T1/A4 dropped. All 7 P0 bugs retired; 280 tests / 82% coverage (was 193 / 81%).`
Owner: @avidullu · Created: 2026-06-24 · Last updated: 2026-06-24
> Lifecycle: `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> Tracking anchors: $7 progress tracker is the source of truth; pointer added to
`docs/README.md`.
> Relation to existing docs: peer-of `docs/DECISIONS.md` (ADR-001, ADR-002); the de-dup work
items (T1/A4) are explicitly gated behind an ADR-001 revisit (see D2).
> Source audit: full findings in agent session 2026-06-24; verified facts mirrored to repo
memory `sports-data-collector-audit.md`.
> Honesty note: every P0/P1 claim below is tagged `[verified]` (checked against current
`master` source) or `[researched]` (needs sign-off). ADR-001 is `[verified]`; the refactor items
that conflict with it are marked ? gated.

```

0. TL;DR

A full code-health audit of `sports-data-collector` surfaced **7** correctness/license-integrity bugs (P0), **10** tech-debt / non-intrusive improvements (P1), and **6** structural/architectural flags (P2). The plan sequences work **safest-first** (tiny, isolated, fully-tested wins ? larger refactors) so we warm up before touching the duplicated per-sport code. The single most important caveat: **items T1 & A4** (the de-dup refactor) conflict with [ADR-001](DECISIONS.md#adr-001--keep-explicit-per-sport-code-instead-of-a-sport-registry-for-now), which deliberately keeps per-sport code ? they are owner-gated behind an ADR revisit, not planned for execution. **The highest-leverage standalone win is B1** (`is\_commercial` not bound to `license\_type`): it is a license-compliance landmine for a product whose commercial-clean licensing is non-negotiable.

1. Problem & goal

`sports-data-collector` is the curated, commercially-licensed **data producer** feeding `badminton-highlight-indexer` (which trains/evaluates the offline rally segmenter that powers `khelsutra.guru`). Bad data or a license-conflation here propagates straight into the training pool and the shipped product.

Goal: retire the correctness and license-integrity bugs, pay down the cheapest tech-debt, and **explicitly record** the structural items the owner must decide on ? without regressions to the ~87%-coverage test suite (`python run\_all\_tests.py`).

Read first: `README.md`, `docs/DECISIONS.md` (ADR-001 + ADR-002), `docs/PIPELINE\_ROLE.md`, `src/core/db.py`, `src/core/schemas/base.py`, `src/parsers/cvat/rally\_cvat.py`.

2. Decisions locked

#	Decision	Source / date	Implication
D1	Execute safest-first. Order = isolation × testability × blast-radius, not raw priority. P0 bugs are interleaved with P1 quick wins so early PRs build confidence.	This plan, 2026-06-24	First ~6 PRs are <50 LOC each, single-file, fully tested.
D2	Honor ADR-001. Do NOT plan the per-sport de-dup refactor (T1, A4) for execution. List it as owner-gated; only revisit if an ADR-001 trigger fires.	ADR-001 [verified]	T1/A4 are tracked but ?; the "Future design sketch" in ADR-001 is the starting point if revisited.
D3	One PR per work item, off `master`, no stacking. Branches named <code><class>-<id>-<slug></code> (e.g. <code>fix-b1-license-validator</code>).	repo convention (CLAUDE.md)	Each row in §7 is one independently-shippable PR.
D4	Every fix ships with a test. Re-run <code>python run_all_tests.py</code> before each PR; suite must stay green.	repo convention	No "fixes only" PRs; tests live next to the change.
D5	P0 license-integrity wins (B1, B4, B5) are sequenced early despite being P0, because they are tiny, isolated, and protect the core commercial-clean invariant.	This plan, 2026-06-24	B1 lands in the first PR cluster.
D6	Docs-only PR first. This doc ships alone (no code) so the owner can re-prioritize before any code lands.	This plan, 2026-06-24	The PR containing this file closes no code items.
D7	`is_commercial` is a STRICT consistency-check validator (NOT a derived <code>@computed_field</code>). It keeps the field but raises hard on any mismatch with <code>license_type</code> against the whitelist ? enforced at the pydantic/schema layer, not just at CLI time. Owner-ratified 2026-06-24 ("enforced very strictly").	Owner, 2026-06-24 (resolves OQ3)	B1 uses <code>@model_validator(mode="after")</code> raising <code>ValueError</code> on inconsistency. A future derive-as-computed-field may follow, but the non-breaking check lands first.
D8	Tier D triage (owner, 2026-06-24). T1/A4 (sport registry) dropped ? ADR-001 stands. A2/A5/A6 promoted to active (independently valuable, no backend needed). A1 (crawler retry) + A3 (read-only DB) parked in <code>docs/NEXT_STEPS.md</code> with explicit criticality triggers ? A1 because the batch-vs-single merge is messy/hard to test thoroughly without the live golden corpus; A3 because it's speculative until a backend exists.	Owner, 2026-06-24	Resolves OQ6/OQ7. Tier D is no longer "flag-only" ? A2/A5/A6 are now executable work items below; A1/A3 are deferred with clear unpark conditions.

3. Foundation ? what was verified vs. what needs sign-off

```
- **[verified]** ADR-001 exists and explicitly keeps per-sport code; its "revisit triggers"
are: sport count > ~8, a 4th+ rally-identical sport, or a cross-cutting feature requiring edits
to every per-sport branch. **None have fired.** ? T1/A4 stay gated.
- **[verified]** Foreign keys ARE enabled (`db.py` `_get_connection`: `PRAGMA foreign_keys =
ON`); the manual cascade in `delete_video` is belt-and-suspenders, not a bug.
53:- B4 ? route both JSON+CSV cricket paths through the existing CSV truthiness helper in
`src/cli/curate.py`.
88:| B4 | `curate.py:_parse_local_annotations` cricket: route JSON path's `wicket` through the
same truthiness helper as the CSV path (`"0"` must be `False`). | ? | No | ? | ? |
103:| T9 | Extract one `DEFAULT_COMMERCIAL_LICENSES` constant; replace the 2-3 copy-pasted
default lists in `validator.py` / `curate.py`. | ? | No | ? |
[#39](https://github.com/Khelsutra/sports-data-collector/pull/39) |
109:| T8b | Type hints for `curate.py` (`Annotation` union alias; replace `List[Any]`). | ? | No
| ? | [#44](https://github.com/Khelsutra/sports-data-collector/pull/44) |
163:- `src/cli/curate.py` ? cricket `wicket` JSON/CSV divergence (B4); `sys.exit` in library
code (A2).
```

26. user (2026-07-02T02:51:02.776Z)

""""Adapter: CVAT rally exports -> our canonical per-rally records.

Why this exists

Our annotation brief (`docs/annotation/ROORA\_BRIEF.md`) asks vendors for a per-rally **CSV** that imports directly via `curate import-local`. A vendor may instead do the work in CVAT and hand back the **CVAT** export: per-frame bounding boxes, with the rally fields (`rally\_number`, `shots\_count`, `ending\_reason` ...) carried as CVAT **object attributes** on each box. The box geometry is irrelevant to us ? it is just a carrier the rater attached the rally metadata to.

This module is the ingestion-side adapter that absorbs that vendor format so the modeling repo never has to. It does two things, both deliberately on **our** side so we fix mechanical problems rather than renegotiating them:

1. **Recompute time from the frame range.** A rally's interval is taken from the first/last **frame** a box for that rally appears on:
`seconds = frame / fps` at the video's true fps. We **ignore** any
`start\_time`/`end\_time` attributes the vendor wrote ? a CVAT job that samples every **step** frames and assumes 30 fps yields systematically wrong times (we have seen `start_time == frame / (step * 30)`, i.e. off by the sampling step at the wrong fps). The frame index is reliable; the vendor's seconds are not.
2. **Surface ? never silently fix ? content defects.** Overlapping rallies, zero-duration rallies, ending_reason values outside the agreed set, and too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected rally boundary would poison ground truth, so that stays a human/vendor call.

Supported input: the single-file CVAT XML export, in either **image mode** (`<image><box>...</box></image>`) or **track/interpolation mode** (`<track><box frame=...></box></track>`). The two PASCAL-VOC / COCO sibling exports encode the same data and can be added later if a vendor sends only those.

from __future__ import annotations

```
import math
import os
import re
import warnings
import zipfile
import xml.etree.ElementTree as ET # noqa: F401 ? ET.Element is the type used in hints
```

Parse vendor-supplied XML/zip with defusedxml to harden against entity-expansion / billion-laughs attacks (docs/CODE_HEALTH_PLAN.md §7 Tier C). defusedxml.ElementTree.Element is a subclass of the stdlib ParseError, so the except clause below is unchanged. The parsed Element objects are ordinary xml.etree Elements, so all .find/.findall calls work as-is.

```
from defusedxml import ElementTree as SafeET
from collections import Counter
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple

from src.core.parsing import safe_int
from src.core.downloader import probe_resolution_fps
```

The agreed cross-sport ending_reason vocabulary for net-separated sports. Kept here as a soft check only (a warning, never a hard failure) so this adapter does not depend on the annotation-spec wording.

```
DEFAULT_ALLOWED_REASONS = {
    "winner",
    "forced_error",
    "unforced_error",
    "service_fault",
    "let",
    "other",
}
```

The observational outcome of the last shot ? orthogonal to ending_reason (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').

```
DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
```

Which court the rally is on, for MULTI-COURT videos (the model's weak spot: background-court pickups are its top false positive). "primary" = the main court the camera is filming; "secondary" = any background court. Blank/absent for single-court footage. Soft check only.

```
DEFAULT_ALLOWED_COURTS = {"primary", "secondary"}
```

Per-rally annotator confidence. "unsure" rallies are surfaced on the review sheet (not errors ? requests for a second look) and can be down-weighted in training. Blank/absent is treated as "sure".

```
DEFAULT_ALLOWED_CONFIDENCE = {"sure", "unsure"}
```

```
def _load_rally_cvat_thresholds(config_path: str = "config/config.yaml") -> Dict[str, float]:
    """Read rally_cvat validation thresholds from config, with documented defaults.
```

```
    Thresholds live in config.yaml under validation.rally_cvat so they're documented
    and overridable without a code change. Defaults match the original hardcoded
    values (see docs/CODE_HEALTH_PLAN.md §7 Tier C).
    """
```

```
    from src.core.config import load_config
    cfg = load_config(config_path)
    vc = (cfg.get("validation") or {}).get("rally_cvat") or {}
    return {
        "min_shots_per_second": float(vc.get("min_shots_per_second", 0.3)),
```

```

    "max_gap_s": float(vc.get("max_gap_s", 15.0)),
    "boundary_tolerance_s": float(vc.get("boundary_tolerance_s", 0.3)),
}

```

CVAT attribute names we map onto canonical columns. Vendor label schemas drift ? suffixed names like "score_before (A-B)", and singular/plural or case/separator variants like "shot_count" (the Roora deliveries) for "shots_count" ? so both these aliases and the incoming attribute names are folded through ``\_fold\_attr\_name`` before matching.

```

_ATTR_ALIASES = {
    "rally_number": ("rally_number",),
    "shots_count": ("shots_count", "shot_count"),
    "ending_reason": ("ending_reason",),
    "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome (in/net/out)"),
    "court": ("court", "court_id", "court (primary/secondary)"),
    "confidence": ("confidence", "sure", "review", "confidence (sure/unsure)"),
    "score_before": ("score_before", "score_before (a-b)"),
    "score_after": ("score_after", "score_after (a-b)"),
    "notes": ("notes", "note"),
}

```

```
def _fold_attr_name(name: str) -> str:
```

27. user (2026-07-02T02:51:14.052Z)

```

32:| D5 | **P0 license-integrity wins (B1, B4, B5) are sequenced early** despite being P0,
because they are tiny, isolated, and protect the core commercial-clean invariant. | This plan,
2026-06-24 | B1 lands in the first PR cluster. |
53:- B4 ? route both JSON+CSV cricket paths through the existing CSV truthiness helper in
`src/cli/curate.py`.
88:| B4 | `curate.py:parse_local_annotations` cricket: route JSON path's `wicket` through the
same truthiness helper as the CSV path (`"0"` must be `False`). | ? | No | ? | ? |
163:- `src/cli/curate.py` ? cricket `wicket` JSON/CSV divergence (B4); `sys.exit` in library
code (A2).
176:- `2026-06-24` ? **Tier S COMPLETE.** T5 merged (#31, green); T8a+T10+B7 merged (#32,
green). Coverage 81%?83% (src) + cleanup.py 0%?79%. **OQ3 resolved ? D7** (`is_commercial` =
strict consistency-check validator, owner-ratified). Status `DRAFT ? IN PROGRESS`. Next: Tier A
(B5, B4, B1).
177:- `2026-06-24` ? **Tier A COMPLETE.** B5+B4 merged (#34); B1 merged (#35). All license-
integrity bugs retired. Tracker refreshed (#33).
===tracker rows around===
| T5 | Replace bare `except Exception` in `src/reporting/__init__.py` with `logging.debug(...,
exc_info=True)` (keep no-raise contract). | ? | No | ? |
[#31](https://github.com/Khelsutra/sports-data-collector/pull/31) |
| T8a | Add type hints to `cleanup.py` + `main(argv)` testability hook (no behaviour change). |
? | No | ? | [#32](https://github.com/Khelsutra/sports-data-collector/pull/32) |
| T10 | `normalize_annotations._to_seconds`: report 4-part (and other unparseable) times in
`NormalizeReport` instead of silent drop. | ? | No | ? |
[#32](https://github.com/Khelsutra/sports-data-collector/pull/32) |
| B7 | `cleanup.py`: track a failure flag, exit 1 if any critical delete failed. | T8a | No | ?
| [#32](https://github.com/Khelsutra/sports-data-collector/pull/32) |

```

Tier A ? License-integrity wins (small, high-value, protect the core invariant)

```

| ID | Deliverable | Depends on | Gated? | Status | PR |
|----|-----|-----|-----|-----|----|
| B5 | `db.py:row_to_video`: wrap enum construction in try/except, log offending
`video_id`+value, map unknown license ? `UNKNOWN` (skip-or-default decision in OQ2). | ? | No |
? | ? |

```

```
| B4 | `curate.py:_parse_local_annotations` cricket: route JSON path's `wicket` through the same
truthiness helper as the CSV path (`"0"` must be `False`). | ? | No | ? | ? |
| B1 | `schemas/base.py`: add `@model_validator(mode="after")` STRICT consistency-check linking
`is_commercial` to `license_type` against the whitelist (raises on mismatch). **OQ3 resolved ?
consistency-check, not derived** (D7). | ? | No | ? | ? |
```

Tier B ? Label-quality wins (small, guard golden labels for the indexer)

```
| ID | Deliverable | Depends on | Gated? | Status | PR |
|----|-----|-----|-----|-----|----|
| B3 | `rally_cvat._detect_boundary_grid`: handle `g == 0` (all boundaries identical) as an
explicit blocker issue. | ? | No | ? | ? |
| B2 | `coachai.py` buffer-collision: when a clamp would invert, **flag + warn** (drop is OQ4);
stop fabricating `+0.001` durations silently. | ? | OQ4 | ? | ? |
| B6 | `badminton_crawler.py`: move per-set CSV cleanup into `try?finally` / `ExitStack` so temp
files can't orphan. | ? | No | ? | ? |
```

Tier C ? Cheap tech-debt (additive, no behaviour change for happy path)

28. user (2026-07-02T02:51:14.854Z)

```
25:#: Used so _parse_local_annotations / _guard_import return concrete types, not Any.
60:     disagreed with the CSV path's explicit truthiness list on the same logical value.
202:         annotations = self._parse_local_annotations(sport_str, vid, parse_path)
283:     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Annotation]:
332:         # B4: route through the same truthiness helper as the CSV path
335:         wicket=_truthy(obj.get("wicket")),
402:         wicket=_truthy(row.get("wicket")),
1208:         sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
```

29. user (2026-07-02T02:51:32.506Z)

```
origin https://github.com/Khelsutra/sports-data-collector.git (fetch)
origin https://github.com/Khelsutra/sports-data-collector.git (push)
https://github.com/Khelsutra/sports-data-collector
===ruff===
ruff 0.15.16
30 F401 [*] unused-import
4 F841 [ ] unused-variable
3 F541 [*] f-string-missing-placeholders
2 E402 [ ] module-import-not-at-top-of-file
Found 39 errors.
[*] 33 fixable with the `--fix` option (4 hidden fixes can be enabled with the `--unsafe-fixes`
option).
```

30. user (2026-07-02T02:51:41.753Z)

help: Remove unused import: `src.core.schemas.cricket.CricketDelivery`

```
F401 [*] `os` imported but unused
--> tests\test_parser.py:1:8
|
1 | import os
|     ^^
2 | import pytest
3 | from src.parsers.badminton.coachai import CoachAIBadmintonParser
|
help: Remove unused import: `os`
```

```
F401 [*] `os` imported but unused
--> tests\test_reporting.py:3:8
|
```



```

1 | """Tests for the collector observability reports. obsreport is a soft dependency;
2 | skip cleanly if it isn't installed."""
3 | import os
  |     ^^
4 | from types import SimpleNamespace
  |
help: Remove unused import: `os`

```

Found 24 errors.

[*] 20 fixable with the `--fix` option (2 hidden fixes can be enabled with the `--unsafe-fixes` option).

===F841 detail===

F841 Local variable `is\_commercial` is assigned to but never used

```

--> src\crawlers\badminton_crawler.py:107:13
  |
105 |         # ShuttleSet is MIT licensed -> commercially friendly
106 |         license_type = LicenseType.MIT
107 |         is_commercial = True
  |         ^^^^^^^^^^^^^^^^^
108 |
109 |         print(f"\n[{idx+1}/{len(matches)}] Processing: {match_name}")
  |

```

help: Remove assignment to unused variable `is\_commercial`

F841 Local variable `rallies\_merged` is assigned to but never used

```

--> src\crawlers\badminton_crawler.py:148:13
  |
146 |         # 2.2 Download & Merge stroke CSV files for this match
147 |         print(" Fetching annotation files...")
148 |         rallies_merged: List[BadmintonRally] = []
  |         ^^^^^^^^^^^^^^^^^
149 |

```

31. user (2026-07-02T02:52:03.778Z)

===pipeline doc heads===

Role in the rally-segmenter training pipeline

This repo is the **data producer** in a two-repo system. Its job is to curate sports videos with per-rally `[start, end)` annotations and commercial-license metadata, then hand that data to the **badminton-highlight-indexer**, which processes, scores, and (Phase 4) *trains* an offline rally segmenter on it.

This doc explains *why* the `export` and `fetch` commands exist and how they fit the bigger picture. The full decision records and end-to-end narrative live in the indexer repo:

- `badminton-highlight-indexer/docs/ROADMAP.md` ? the four-phase plan and end-state.
- `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003 (acquisition policy), ADR-004 (offline learned segmenter).

Why this repo matters

The indexer's best rally mode refines local detections with a **paid Gemini API**. The strategic goal is to replace that with an **offline model trained on our own commercially-licensed, annotated corpus** ? making rally detection free, offline, and self-improving. None of that is possible without a clean, licensable, accurately-labelled, full-resolution dataset. That dataset is what this repo produces.

What the two key commands do (and why)

===config===

Sports Data Collector Configuration

Path to the SQLite database

database_path: "../data/sports_metadata.db"

Storage configuration for downloaded videos

```
video_storage:
  # Folder where downloaded or local videos will be stored
  local_dir: "../data/videos"
  # Maximum allowed size for automatic downloads (in Megabytes)
  max_download_size_mb: 12000
  # Whether to attempt video downloads automatically during ingestion
  auto_download: true
  # Optional cap on download resolution height (e.g. 1080 to avoid pulling 4K).
  # Leave unset/null to always fetch the best resolution available.
  max_height: null
```

Whitelist of licenses considered safe for commercial use

```
commercial_licenses:
  - "MIT"
  - "Apache-2.0"
  - "BSD"
  - "CC0"
  - "CC-BY"
  - "Public-Domain"
  - "Proprietary" # Used for custom user-added local videos
```

Ingestion limits

```
ingestion_limits:
  max_total_download_size_mb: 60000
  max_total_video_count: 10
```

Whitelist of all sports supported in the system

```
supported_sports:
  - "badminton"
  - "tennis"
  - "tabletennis"
  - "pickleball"
  - "cricket"
```

Validation / parsing thresholds. Each has a rationale comment; override here only with a deliberate reason. See docs/CODE_HEALTH_PLAN.md §7 Tier C.

```
validation:
  coachai:
    # Seconds subtracted from the first stroke (serve) to get the rally start.
    # 1.0s covers the serve wind-up without eating the previous rally's tail.
    start_buffer: 1.0
    # Seconds added to the last stroke to get the rally end. 2.0s covers the
    # shuttle settling / umpire call after the last shot.
    end_buffer: 2.0
  rally_cvat:
    # Rallies with fewer than this many shots/second are flagged "shot_density"
    # (medium) ? the span may be inflated by a stray box. 0.3/s ? a shot every
    # ~3.3s, well below any real rally tempo.
    min_shots_per_second: 0.3
    # A gap between consecutive rallies longer than this is flagged
    # "possible_missed_rally" (medium). 15.0s catches an omitted rally that left
    # a visible hole without false-firing on normal between-rally breaks.
    max_gap_s: 15.0
    # The ±boundary bar (seconds) rally first/last boxes must meet. 0.3s is the
    # golden-quality tolerance from Brief §4; a coarser sampling grid makes it
    # structurally unmeetable (the boundary_grid_snap blocker).
```

boundary_tolerance_s: 0.3

32. user (2026-07-02T02:52:17.667Z)

```
ls: cannot access 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/DECISIONS.md': No such file or directory
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/ROADMAP.md
src/cli/curate.py:36:      catch and handle it. See docs/CODE_HEALTH_PLAN.md §7 Tier D (A2).
Binary file src/cli/__pycache__/curate.cpython-313.pyc matches
src/core/db.py:181:      See docs/CODE_HEALTH_PLAN.md §7 Tier C.
src/core/manifests.py:9:See docs/CODE_HEALTH_PLAN.md §7 Tier D (A5).
Binary file src/core/__pycache__/db.cpython-313.pyc matches
Binary file src/core/__pycache__/manifests.cpython-313.pyc matches
src/parsers/badminton/coachai.py:19:      and overridable without a code change (see
docs/CODE_HEALTH_PLAN.md §7 Tier C).
Binary file src/parsers/badminton/__pycache__/coachai.cpython-313.pyc matches
src/parsers/cvat/rally_cvat.py:45:# billion-laughs attacks (docs/CODE_HEALTH_PLAN.md §7 Tier C).
defusedxml.ElementTree
src/parsers/cvat/rally_cvat.py:90:      values (see docs/CODE_HEALTH_PLAN.md §7 Tier C).
===README doc links===
```

Documentation

- docs/PIPELINE_ROLE.md ? how this repo feeds the **badminton-highlight-indexer** (``export` / `fetch``).
- docs/DECISIONS.md ? collector-local decision records (**ADR-001**: keep explicit per-sport code instead of a registry, with an "adding a new sport" checklist).
- docs/annotation/ ? the vendor annotation package used to produce golden rally labels:
 - [ROORA_BRIEF.md](docs/annotation/ROORA_BRIEF.md) ? vendor brief + the canonical rally CSV schema.
 - [ANNOTATION_GUIDE.md](docs/annotation/ANNOTATION_GUIDE.md) ? the illustrated rater guide (singles & doubles).
 - [INTAKE_MANIFEST_TEMPLATE.csv](docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv) ? per-video intake sheet.
 - [NDA_TEMPLATE.md](docs/annotation/NDA_TEMPLATE.md) ? mutual NDA template (have counsel review).

Running Tests

We use ``pytest`` for unit and integration testing:

```
```bash
python -m pytest tests/
python -m pytest tests/ --cov=src --cov-report=term-missing # with coverage (~87%)
```
```

33. user (2026-07-02T02:53:18.299Z)

```
ci.yml
9f1e792 2026-06-24 fix(rally_cvat): harden XML parsing with defusedxml (T7/OQ5) (#42)
ls: cannot access 'run_all_tests.py': No such file or directory
15:      python-version: "3.11"
21:      run: python -m pytest -q
```

34. user (2026-07-02T02:53:54.704Z)

```
13:- `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003
59:artifacts/
136:python -m pytest tests/ --cov=src --cov-report=term-missing # with coverage (~87%)
23:      parser.add_argument("--video-url", default="https://youtube.com/watch?v=example",
help="URL to the video")
185:      print(f" Warning: Failed to fetch Set {s} CSV: {e}")
```

35. user (2026-07-02T02:55:03.570Z)

Structured output provided successfully